

T - L - I)

15 20 10

(marks)

__/__/__

Operating Systems

Topics

- i) OS Introduction
- ii) Linux Introduction
- iii) Linux Arch. and Structure
- iv) Basic commands / Advanced Commands.
- v) vi / vim Editor
- vi) Linux Shell Programming (Shell scripts)
- vii) Process Management
- viii) Memory Management

* Why there is a need of OS ?

- Computer is a machine/hardware does tasks differently & accurately for user.

Basic Functions of computer

- Data Storage
- Data Processing
- Data movement
- Control

- Any user can't directly communicate with hardware, so we need to have an interface b/w user & hardware, which is provided by Operating System (OS).

⌘ End User

Software Applications (VIC, IDE, chrome)

Operating System

Hardware Resources

(CPU, RAM, HDD, keyboard, monitor)

↓
Primary input device

* By default there is atleast 1 bootable device on our machine and that can be our (hdd or ssd).

- OS is an interface between end user and hardware resources.
- OS also acts as a resource manager or resource allocator
- OS is also a control program

From an end-user perspective:-

- it is a bootable ^{Storage} device (CD/DVD/USB/SSD/HDD).
- * OS is a Software, i.e. it's a collection of programs.

OS $\Rightarrow$ Kernel + Utility sw + Application sw
(Core OS) (Control panel, disk manager, etc.) (Paint, notepad, etc.)

* Kernel Functions

- ① Process Management
- ② CPU Scheduling
- ③ Memory Management
- ④ File and I/O Management
- ⑤ Hardware Abstraction
(Extra utility functions)
- ⑥ User Interfacing
- ⑦ Networking
- ⑧ Security and Protection

Embedded
device

PC

.so → Stored Objects

* PDF is also a binary file

* Program

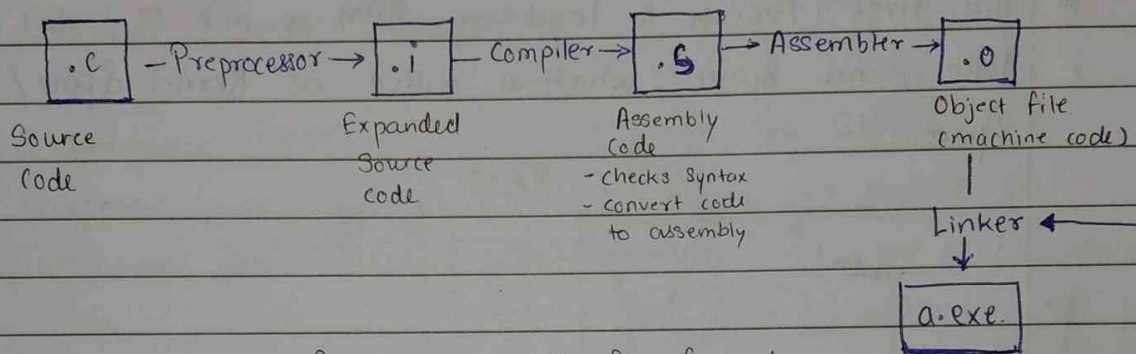
- Set of instruction

- Present in hard disk

Source code
(.cpp / .c / .java)

Executable code
(.exe / .out)

→ Compilation flow



.exe and .out files have diff. file format.

.exe → Windows → PE (Portable executable) (MZ)

.out → Linux → ELF (executable linking format) (ELF)
file

.dll/.so
Files
lib

Structure of .out / .exe file

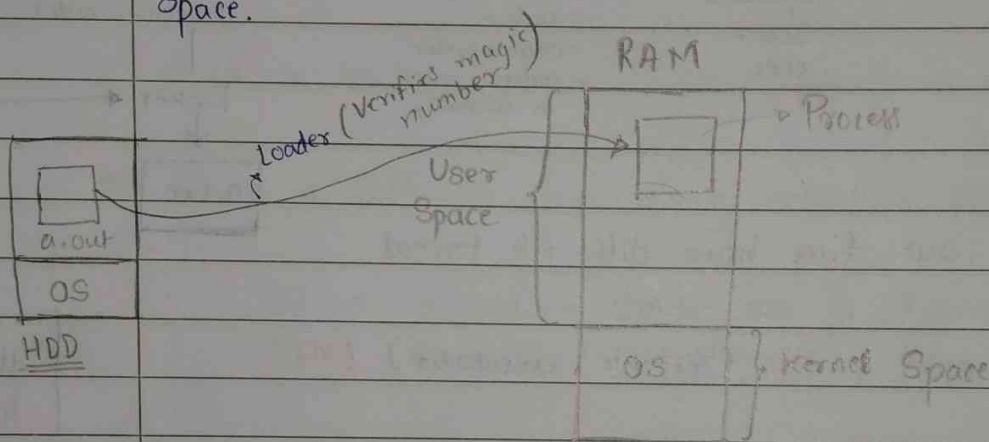
.out		
0	Primary Header	① Primary Header :- address of entry point function is kept over here,
1000	BSS	- information about other sections
500	Data	- it consists a magic number
1000	RO	(magic number is file format specific number)
1200	Code / Text	② BSS :-
1600	Symbol Table	- contains uninitialized global & static variable
2000		③ Data:
		- contains initialized global & static variables
		④ RO (Read Only Section):
		- constant character and string literals

⑤ Code/Text Section :- Executable instructions

⑥ Symbol Table :- (for debugging purposes)

- information about function ~~activation~~ and variables.

* The first process to load on RAM is our OS, and it takes some space, which is called as Kernel Space/System Space.



Process

Stack	→ FAR
Heap	→ Dynamically created obj / vars.
Data	
Code/Text	
RO	→ Read Only

PCB (Process Control Block)

- stores important information related to process.
- it is a structure created by the Operating System.
- Info inside PCB

- Pid
- PPid
- File System Info.
- Memory Info
- CPU Scheduling
- Exit Status.

* User Interface

- UI of OS is also a program (Shell) that acts as interface between end user and kernel.

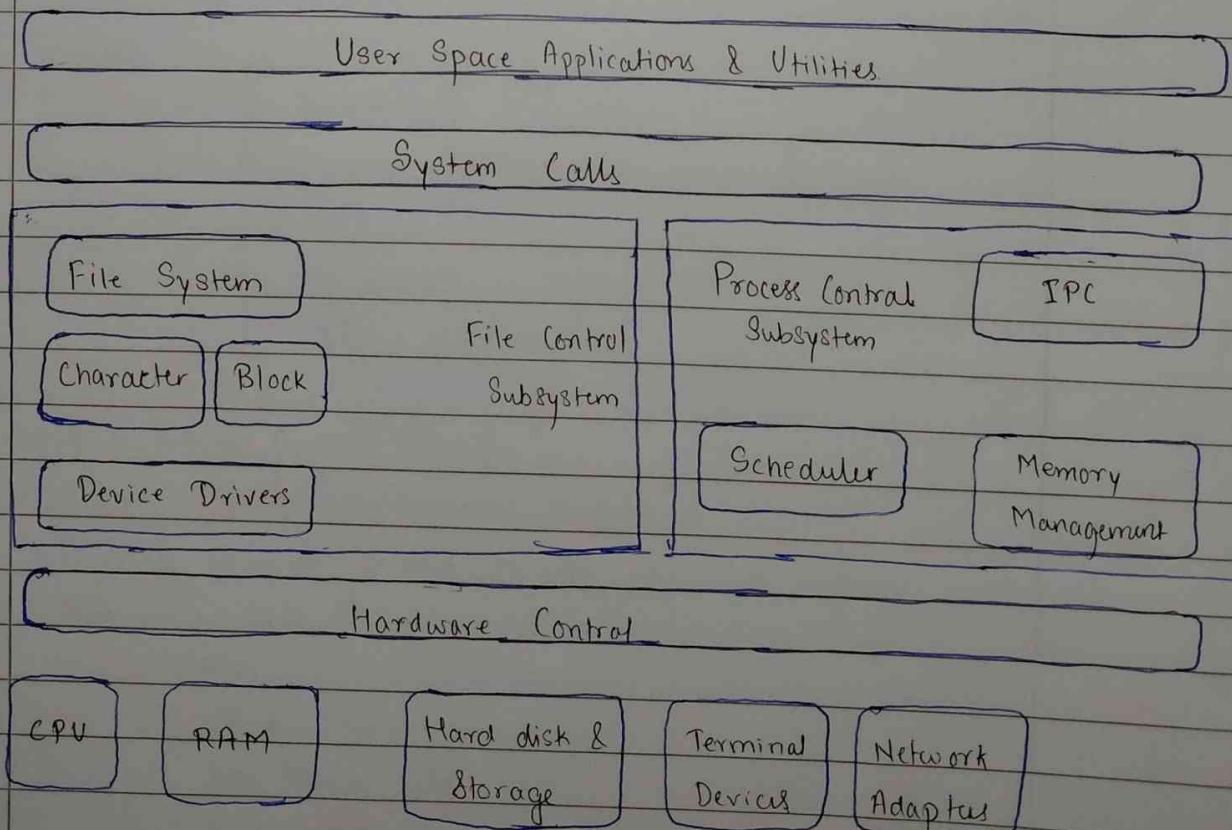
Shell $\Rightarrow$ Command Interpreter

- User Interfacing (Shell) : Linux $\rightarrow$ GNOME / KDE
 - Graphical user interface (GUI) : windows $\rightarrow$ explorer.exe
 - Command line interface (CLI) : windows $\rightarrow$ cmd.exe, powershell
Linux $\rightarrow$ bash shell, zsh, rbash
↓
ksh, csh, etc.
borne again Shell

* UNIX Operating System

- UNIX :- UNICS (Uniplexed Information & Computing Services / System)
- Developed at AT&T Bell Labs in US, in the decade of 1970's
- Unix is the First Multi-User, Multi Programming, & Multi-Programming System.
- Unix was specially designed for developers by developers.
- In Unix, whatever that can be stored is considered as a File and whatever is active is referred as Process.
- File has Space & Process has Life.

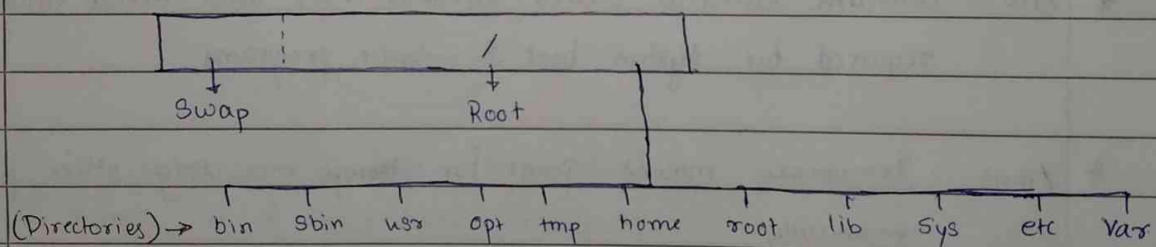
UNIX
Kernel
Arch →



- Default Size of block is 4 kb.
- Minimum block size is 512 byte.

* Linux

- Linux is a family of open-source, UNIX-like Operating Systems based on Linux kernel (Linus Torvalds).
- Name of kernel file of Linux is vmlinuz.
- Linux File System Structure
- here we have two partitions
 - ① Swap area
 - ② Root

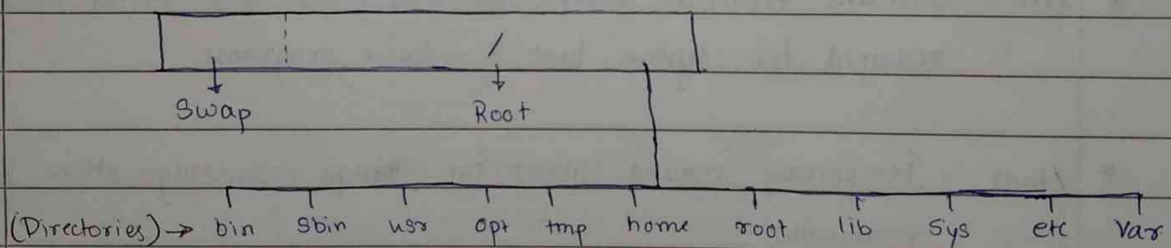


- Linux follows FHS (File System Hierarchical Standards).
- In Linux, we have 2 types of Path
 - ① Absolute Path - path given from the root directory
 - ② Relative Path - path given relative to current directory
- The Linux kernel is present inside the boot directory
- grub - grand unified boot loader

- Default Size of block is 4 kb.
- Minimum block size is 512 byte.

* Linux

- Linux is a family of open-source UNIX-like Operating Systems based on Linux kernel (Linus Torvalds).
- Name of kernel file of Linux is vmlinuz.
- Linux File System Structure
- here we have two partitions
 - ① Swap area
 - ② Root



- Linux follows FHS (File System Hierarchical Standards).
- In Linux, we have 2-types of Path
 - ① Absolute Path - path given from the root directory
 - ② Relative Path - path given relative to current directory
- The Linux kernel is present inside the boot directory.
- grub - grand unified boot loader

- * /bin :- Contains user commands in binary format, e.g. cp, mv, ls, cat, etc.
- * /sbin :- Contains admin/system commands in binary format, e.g. adduser, lscpu, deluser, mkfs, etc.
- * /home :- contains dirs of all users for multiuser system
- * /root :- root is a home directory of root user
- * /etc :- contains host specific system configuration files.
- * /dev :- Contains device files for all hardware devices (e.g. char & block devices)
- * /lib :- Contains essential shared libraries (.so) and kernel modules (.ko) required for system boot & basic programs.
- * /mnt :- Temporary mount point for ~~tempo~~ mounting other file systems manually.
- * /opt :- Contains optional or third party software packages.
- * /proc :- provides information about system processes and kernel.
- * /sys :- Used for device management and kernel settings
- * /tmp :- temporary files; contents are usually cleared on reboot.
- * /usr :- Contains user installed software packages and shared read-only data.

* File

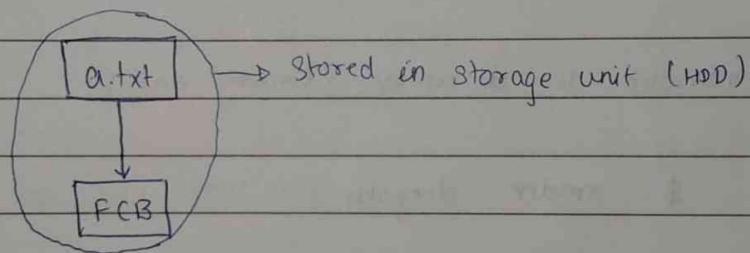
- User view:-
 - i) Named collection of logically related information / data.
 - ii) collection of character / records / lines.
 - iii) basic storage unit

- System view:- File is a stream of bits / byte.

File = data + metadata

{ data - contents of file
metadata - info about the file

- FCB/inode:- File Control Block



- * - it contains info like, type of file, size: no. of bytes, links: no. of hard links, mode (permissions), user & group, info about data blocks.

- * Create a file and insert data into it at time of creation we use

~~cat~~ \$ cat > a.txt

- * To get FCB/inode information of a file we use

\$ stat a.txt.

- * To append data to a file

\$ cat >> a.txt.

* To change shell from bash to any other shell we use chsh command

- Touch command can be used to

- ① Create a new empty file
- ② Update the timestamp of a file

- If we move a file from same dir to same dir we can rename a file.

\$ mv a.txt aa.txt → Renamed file

- Copy a file from one path → another

\$ cp file_name dest_folder_name.

- remove a directory (empty only)

\$ rmdir dirpath.

- change dir - home dir - Previous dir

\$ cd path

\$ cd ~

\$ cd -

- list directory contents

\$ ls

* To get specific ~~exam~~ info about a file, we use.

\$ stat -c %F a.txt → Gives us the type of file

\$ stat -c %i a.txt → Gives us the inode number of file.

* Types of files in Linux (There are 7)

① Regular File (-) ^{Symbol}
.txt, .pdf, .out, .mp3, .mp4

② Directory file (d) → (folder)
directory / folder

③ Block file (b) →
Storage devices (usb, hdd, ssd, etc)

④ Character ~~Device~~ file (c)
Character Devices (mouse, keyboard)

⑤ Socket file (s)
- files related to networking

⑥ Pipe file (p)

⑦ Link file (l)
- shortcuts.

* Pipe and Socket files are used in IPC (Inter-process communication)

* **chown** --> change file owner/group file.

\$ sudo **chown** root : root file.name.
 username group name

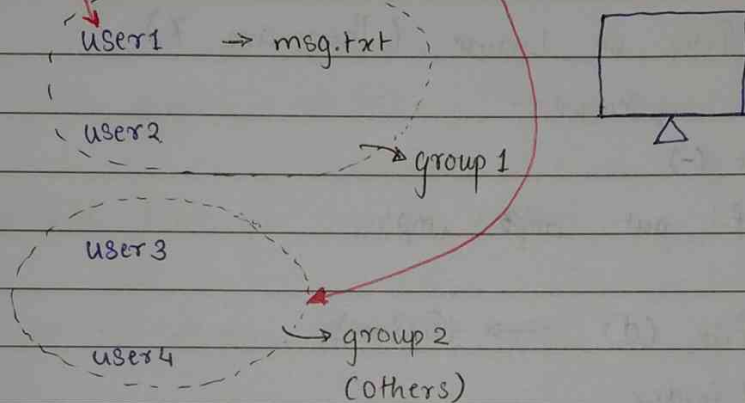
__/_/_

* **Permissions**

\$ **d rwx rwx rwx**
 ↑
 file type

user **group** **other**
 Permissions (read, write, execute)

*



* **Change file permissions for user/group/others**

\$ **chmod** **+rwx** file.name
 -rwx file.name

] --> grant/revoke permissions of all (user/group/others)

- to set permissions of only user we use binary rep,

\$ **chmod** **0700** msg.txt --> user only

\$ **chmod** **0770** msg.txt --> user & group

i.e. 0 7 7 0
 (1 1 1) (1 1 1) (0 0 0)
 rwx rwx - - -

So final permissions are (-rwxrwx---

(only Octals are allowed 0-7)

__/__/__

- Provide rw & wx & r only to user & group & other respectively.

\$ chmod 0634 msg.txt

$\begin{matrix} \swarrow & \searrow & \searrow \\ (010) & (011) & (100) \\ rwx & rwx & rwx \end{matrix}$

- * Per file one FCB / inode gets created, i.e. no. of inodes = no. of files on disk.

* File System

way to store data onto the disk in an organized manner so that it can be accessed efficiently & conveniently.

UNIX :- UFS (UNIX file system)

Linux :- Extended File System ext2, ext3, ext4, etc.

Windows :- FAT, NTFS, etc.

Mac :- XFS, HFS, ReiserFS, etc.

File System Structure

Boot Block/ Boot Sector	Super Block/ Volume Control Block	inode List/ Master File Table	Data Block
----------------------------	--------------------------------------	----------------------------------	------------

- Data Block :- contains the blocks of data
- inode List :- this contains the inode info. of all the data blocks.
(stores FCB)
- Super Block :- this contains used block, free block, inode block information
- Boot Block :- this has Bootstrap and Boot Loader.

* Creating file system on

① Windows

Format.exe - utility used to format a partition

② Linux

mkfs - make file system command

ex:- ~~mkfs~~

```
$ sudo mkfs -t ext3 /dev/sdb1
```

```
$ sudo mkfs -t vfat /dev/sdb1
```

-t → <type> specifies the file system type

common types → ext2, ext3, ext4, ntfs, vfat, xfs, etc.

→ To make a file hidden

```
$ touch .file
```

↑
hidden

(to see hidden files → \$ ls -a)

(imp) → Manual Page has 9 sections. (man command)

- _/_/_
- Read first n or 10 no. of lines of a file.

\$ head a.txt → gives 1st 10 lines

\$ head -n 13 a.txt → gives the top 13 lines

- Similarly, we have tail to read content of file from bottom/last.

\$ tail a.txt → gives 10 lines from bottom

\$ tail -n 2 a.txt → gives 2 lines from bottom.

- We have Sort command,

\$ sort a.txt

\$ sort -n a.txt → sorts on basis of numerical data present inside.

\$ sort -r a.txt → reverse order.

- we have uniq command. (filters adjacent matching lines from file)

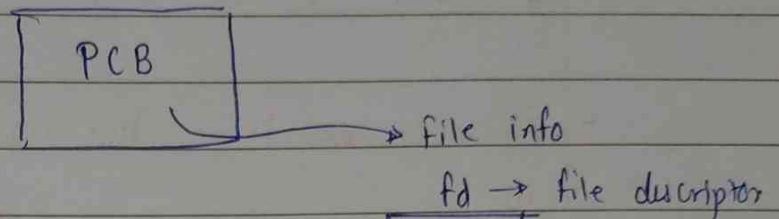
\$ uniq -d a.txt → gives duplicate items

\$ uniq -u a.txt → gives unique items

Redirection

By default every program opens 3 files :-

- ① Stdin
- ② Stdout
- ③ Stderr



3 types of Redirection :-

- ① Input Redirection
- ② Output — " —
- ③ Error — " —

stdin	→	0
stdout	→	1
stderr	→	2
student.db	→	3
data.db	→	4

Input Redirection

Reads input from the ~~terminal~~ file instead of stdin

Command < file

to do redirection "<" Symbol is used.

Output Redirection

Write stdout to file instead of the terminal, i.e. o/p will be written in file instead of stdout

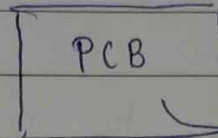
To do output redirection ">" or ">>" symbol is used

Command > file
or
Command >> file

Redirection

By default every program opens 3 files :-

- ① Stdin
- ② Stdout
- ③ Stderr



fd → file descriptor

3 types of Redirection :-

- ① Input Redirection
- ② Output — " —
- ③ Error — " —

stdin	→	0
stdout	→	1
stderr	→	2
student.db	→	3
data.db	→	4

* Input Redirection

- Reads input from the ~~terminal~~ file instead of stdin

Command < file

- to do redirection "<" symbol is used.

* Output Redirection

- Write stdout to file instead of the terminal, i.e. o/p will be written in file instead of stdout
- To do output redirection '>' or '>>' symbol is used

Command > file
or
Command >> file

* Error Redirection

- Error will be written into a file instead of stderr.
- To do error redirection we use `2>` or `2>>` symbol.

ex:- `ls -xyz` → gives error

`$ ls -xyz 2> err.txt`

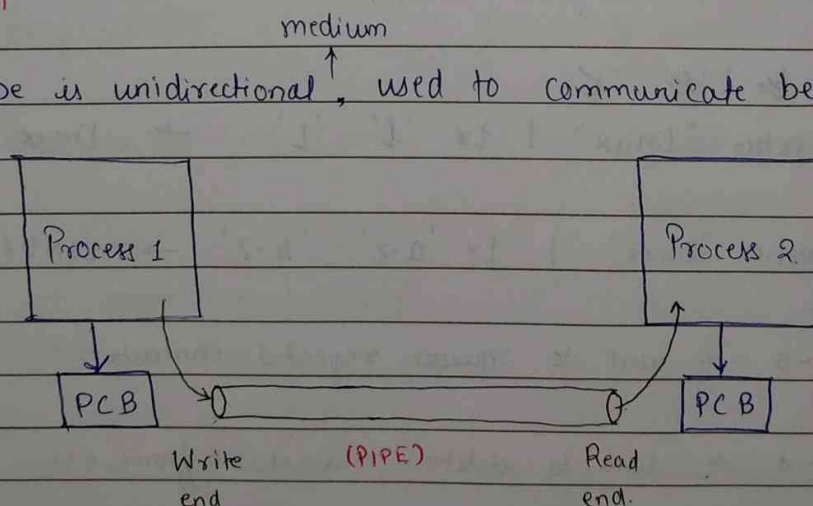
- * To do both stdout & stderr at same time we use,

`$ ls -l > out.txt 2> err.txt`
if no err if err

- | | |
|-------------------------------------|-------------------|
| Command <code>2></code> file | → overwrite file |
| or | |
| Command <code>2>></code> file | → append to file. |

* Pipe (IPC)

- Pipe is unidirectional, used to communicate between process



cmd1 | cmd2
↑
i/p

(O/P of cmd1 goes as an i/p to cmd2)

- 1

*

6

✱

- 1

ex:-

Or

5-11

* ? or * → works as wild card in terminal

→ e.g. a* → any file starting with a & having any character after

e.g. D?c → any file having 1st letter D & last letter c and middle 1 character only can be anything

* To create or remove file ~~from~~ name in given range, we do.

\$ touch file{1-100}.txt → will create file1, ..., file100, etc.

\$ rm file{1-100}.txt → will remove all the above files

* Grep Command (GNU reduced expression parser)

- \$ grep "is" a.txt → o/p's pattern in file (highlighted)

- \$ grep -w "is" a.txt → o/p's only words (independent) like pattern

- \$ grep -w -i "is" a.txt → -i will ignore case-sensitivity.

-w → word

-i → ignore case sensitivity

-n → gives line no.

- \$ grep -n "is\$" a.txt → gives line which ends with (is) / pattern.

- \$ grep -n "^is" a.txt → gives line which starts with (is) / pattern.

* In grep we use, "." instead of "?".

```
$ grep -ni "d.c" a.txt
```

* to specify allowed characters in pattern we use

```
$ grep -ni "d[aeo]c" a.txt
```

```
$ grep -ni "d[^aeo]c" a.txt
```

```
$ grep -ni "d[aeo$]c" a.txt
```

* extended grep

* (+) → 1 or more occurrences

e.g. `egrep "da+c" a.txt`

* (?) → 0 or 1 occurrence

e.g. `egrep "da?c" a.txt`

↑
occurrence of a.

* ({n}) → n occurrences

e.g. `egrep "da{2}c" a.txt`

`egrep "da{3,}c" a.txt` → at least 3 occurrences

`egrep "da{,4}c" a.txt` → maximum 4 occurrences

`egrep "da{2,4}c" a.txt` → min 2 & max 4

(occurrences of a)

* Find Command

- helps you search for files and directories in real time, based on type, size, name, time, permissions.

Ex:- ① name

② file

find . -name "a.txt"

find . -type f ".txt"

③ dir

④ Permissions

find . -type d

find . -type f -perm 0755

⑤ size

find . -size +4K → file having size $\geq 4kb$

* Command Nesting

① cmd1 | cmd2

② cmd &

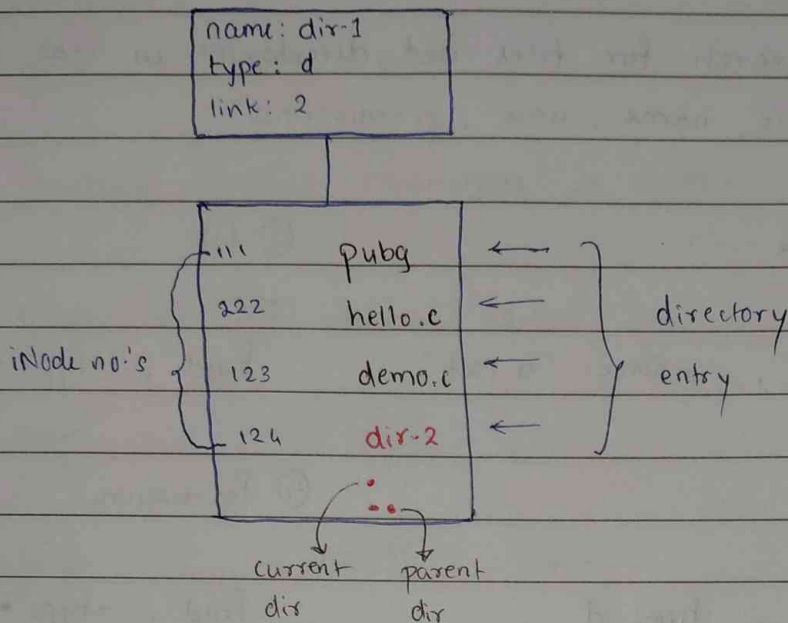
- command is executed in background.
- also called as asynchronous.

- ex:- sleep 10 &

③ cmd1 ; cmd2

- commands are executed separately.

* Links



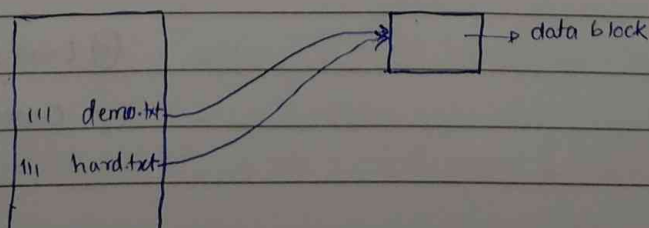
- The link count is 2 because, while in current directory, it has a link to its ~~previous~~ own directory using "." and in its parent directory we have the entry there also, so there are 2 links.

• 2 types of link

① Hard Link

In demo.txt hard.txt

- Here the system ~~creates~~ creates a new entry entry in the directory having the same inode, just with a different name.



(Imp) - So, even if we delete the original entry of a file, then also we can access the file using the hard link we created.

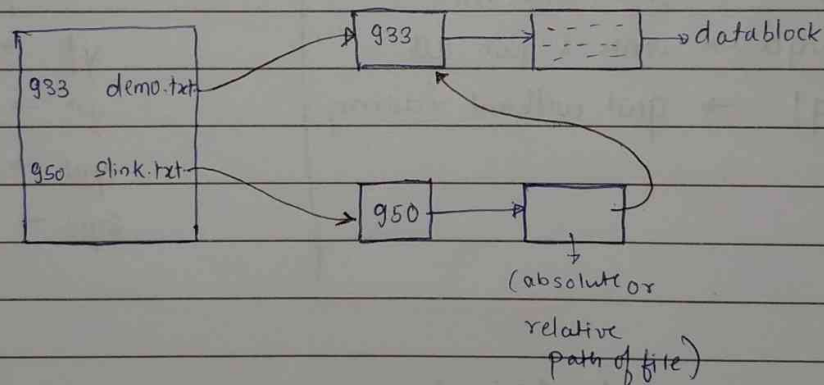
- Hard links cannot be created for directories, or across diff. filesystems.

- If changes are done using the hard link, the changes reflect in the original file too.

② Symbolic link

→ Command → `ln -s filepath destinationpath`

e.g. `ln -s demo.txt slink.txt`



- The symbolic link gets a new inode number and it stores the path (absolute or relative) of the target file.

- If the target/Original File is deleted, then we cannot access the file anymore using the ~~the~~ Symbolic link.

- Here, a new file gets created ^{with different} ~~so the~~ inode no, so the link count doesn't get incremented.

- Can link to file or directories
- Points to the path (name), not the data
- If the original is deleted, the symlink breaks

* Vim Editor (Bill Joy) → designed vim editor

2 modes :- ① Command mode (default) (esc)
② Insert mode (i)

→ insert instructions in .vimrc file to get intellisense
\$ vim ~/.vimrc

Quit :-

:q → quit
:wq → ^{write} ~~save~~ & quit
:w → write into file
:wqa → write & quit all.
:q! → quit without saving

Copy → yy → copy current line
4yy → copy 4 lines from current
:3,5 y → copy line 3 to 5
y\$ → copy line after cursor
y^ → copy line before cursor
yw → copy current word
2yw → copy 2 current word
from cursor

Cut :-

x → delete single char
dd → cut current line
3dd → cut 3 lines from current line

(Similarly ~~for~~ other commands are like copy commands, just use 'd' instead of y)

Paste :- P

Undo :- u

Redo :- ctrl + r

* Find and Replace

① :s / pattern1 / pattern2

find replace

→ Find and replace first occurrence of current line

② :s / pattern1 / pattern2 / g

→ find and replace all occurrence of current line

③ :%s / pattern1 / pattern2

- Find and replace all occurrences of in file

Shell Scripting

- collection of commands along with programming constructs.
- script will differ from shell to shell
- scripts are interpreted by shell (interpreter line-by-line)
- Speed is slower

- comments in shell begin with "#"

- echo command

-e → enable escape sequences

-n → no newline after echo

- Run the script → make it an executable using

> chmod +x demo.sh

execute → > ./demo.sh

* Or directly use bash demo.sh command directly, without giving

* Bash file doesn't have a header, it has a shebang line, i.e.

1 #!/bin/bash

* escape sequences :-

-e → enables escape sequences

e.g. echo -e "DAC \n OMC"

echo -e "DACC \b OMC"

→ Generalised format

declare variable.

Syntax

var-name = data

Read the value of variable

Syntax

var-name or {var-name}

example:-

echo -e "Num1 val"

num1 = 10

→ stores 10 in num1

num1 = {num1}

→ prints (num1 = 10)

str = "sunbeam"

PI = 3.14

echo "PI : {PI}"

echo "str : {str}"

★ → To read value from end user, we use **read** command

ex:-

echo "enter value"

read val

echo "entered value : {val}"

→ To perform arithmetic operations, we use **expr** command

for ex:- **expr 10 + 2**
→ 12

expr 10 - 2
→ 8

(for multiplication we have to suppress its special meaning

(Imp) **expr 10 /* 2**
→ 20

(but "/" division only outputs integer values so, we need to specify its precision using **scale** command inside **bc** command)

echo "scale=2; 10/3" | bc

↩
Pipe the o/p to bc = basic calculator.

Example

#!/bin/bash

echo "Enter n1"

read n1

echo "Enter n2"

read n2

res=`expr \$n1 + \$n2`

echo "result : \$res"

→ or **(res = (expr \$n1 + \$n2))**

→ Calculate the area of circle

```
#!/bin/bash
```

```
echo "Enter rad: "
```

```
read rad
```

```
PI=3.14
```

```
area=`echo "scale=2; $PI * $rad * $rad" | bc`
```

```
echo "area: $area"
```

* if - else in bash script

- for condition, using test Command

logical operator:-

! → !

< → -lt

&& → -a

<= → -le

|| → -o

= → -eq

> → -gt

!= → -ne

>= → -ge

Syntax for if only

```
if [cond]
```

```
then
```

```
----
```

```
fi
```

if - else only

```
if [cond]
```

```
then
```

```
----
```

```
else
```

```
----
```

```
fi
```

if, elif, else .

```
if [cond]
```

```
then
```

```
----
```

```
elif [cond]
```

```
then
```

```
----
```

```
else
```

Example

```
if [ `expr $num % 2` -eq 0 ]
then
    echo "num is even"
else
    echo "num is odd"
fi
```

* to check if file exists or not

check if file exists or not
↓

```
if [ -e $Path ]
```

then

```
    echo "Valid Path"
```

else

```
    echo "Invalid Path"
```

fi

→

```
echo "..."
```

```
if [ -f $path ]
```

```
    echo "Regular file"
```

else

* Switch - Case

Syntax

```
case $var in
```

```
1) Statement
```

```
;; → break
```

```
2) Statement
```

```
;;
```

```
*) default case
```

```
;;
```

```
esac
```

* Loops

① While

#Syntax

<init>

while [cond]

do

<modif>

done

Ex:-

i=1

while [\$i le 5]

do

echo "\$i"

i=`expr \$i + 1`

done

② Until

#Syntax

<init>

until [cond]

do

<modif>

done

--> iterates till condition is false

③ For

#Syntax

Ex:-

for var in collection

do

done

for var in 1 2 3 4 5

do

echo "var is \$var"

done

or for var in {1..10}

→ to print every file inside a dir

```
for var in `ls`  
do  
    echo "$var"  
done
```

→ Parameters (Positional)

```
./demo.sh  arg1  arg2  arg3  arg4  
$0         $1    $2    $3    $4
```

access all args: \$*

count all args: \$#

* At a time we can only access 9 arguments, to access other arguments we use **shift**

* Shift N command is used to skip N parameters from left
N+1 becomes \$1
N+2 becomes \$2

* Function

① not accept not return

#Syntax

```
function fun-name ()  
{  
    .....  
}
```

```
function msg ()  
{  
    echo "Hello"  
}
```

Calling the function

Fun-name

msg

→ fun call

2) Pass value but not return

#Syntax

```
function sum ()  
{  
    # $1 → arg1  
    # $2 → arg2  
    res = `expr $1 + $2`  
    echo "res : $res"  
}
```

#function call

```
sum 10 20
```

3) Pass a value and return ~~the~~ value

#Syntax

```
function mul ()  
{  
    res = `expr $1 /* $2`  
    echo "$res"  
}
```

→ used to return values.

~~res = \$(mul 10 2)~~

res=\$(mul 10 2)

echo "res : \$res"

* We also have a **return** keyword ^{exit or} which returns from the functions, and not execute later statements.

* arrays

#Syntax

arr = (10 20 30 40 50)

→ To print all the values :- `${arr[*]}`

To print individual element :- `${arr[i]}`

To print the # of elements in array :- `${#arr[*]}`

→ To add an element in arr :- `arr+=(60)`

→ declare an array - declare -a arr

* String Operations

* `str='sunbeam'` or `str="sunbeam"`

* Concatenate

`str1='AB'`

`str2='DE'`

`str3="$str1$str2"`

→ `str3='ABDE'`

* `-n STRING` → length of string

* `STRING1 = STRING2` → strings equal check

* `STRING1 != STRING2` → strings not equal check

→ read 2 strings at a time

```
read str1 str2
if [ $str1 = $str2 ]
then
    echo - - - -
else
    echo . - - - -
fi
```

→ reverse a string & store in variable

```
Str1=`echo "$str1" | rev`
```

∴ now we can check if string palindrome or not.

→ Convert to uppercase

```
str='sunbeam'
```

```
echo "str: ${str^^}"
```

```
echo "str: ${str^^}"
```

converts only 1st char to uppercase.

→ Convert to lowercase

```
echo "str: ${str,,}"
```

```
echo "str: ${str,,}"
```

convert only 1st char to lowercase

→ Check for a 10 digit num / using grep with strings

Starting with [^] ... \$ ending with

{n} → exact n char/digits

[a-b] → range of allowed char / digits

if [[\$num =~ ^[0-9]{10}\$^{***}]]

then

echo "Valid num"

else

echo "Invalid num"

← conditions need to be enclosed in square brackets

→ The .bashrc file is executed everytime we create a new instance of the terminal and for every .bashrc running instance, only one .profile file is executed/run, which is run when we login to the system.

→ Classification of OS

1) Mainframe 2) Desktop 3) Multi-Processor

4) Distributed 5) Hand-held 5) Real-time

* Read other units/sections from PDFs/class notes.

(Regarding Process/Memory management)

Software Development Methodologies _/_/_ (Deployment on cloud)

- SDLC
- SCM
- Agile
- Containerization
- Testing
- Containers Orchestration
- DevOps
- CI/CD pipeline

* Deployments :-

- 1) Traditional - using physical machines
- 2) Virtual - using VMs
- 3) Containerized - using containers (Docker)

* Software Engineering

Software $\rightarrow$ Package

Program
Libraries
Resources
Config files

Building Tools

- $\rightarrow$ Ant (deprecated)
- $\rightarrow$ Maven
- $\rightarrow$ Gradle

- Software is more than just a program code.
- A program is an executable code, which serves some computational purpose
- Software is considered to be collection of executable programming code, associated libraries and resources.

- Software Engineering

- Software Engineering is an engineering branch associated with development of software product, using well-defined scientific ~~para~~ principles

- Why SE is important ?

- To build complex systems in timely manner
- Ensures high quality of software
- Imposes discipline to work
- Minimizes software cost
- Decreases time

- Characteristics of Good Software

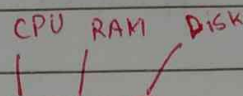
- Operational
- Transactional
- Maintainence

- Operational can be measured by :-

- Budget → developed within budget
- Usability → should be usable by end user
- Efficiently → efficiently use storage and space
- Correctness → provide all functionalities correctly without bugs / issue
- Functionality → meet all requirements of end user
- Dependability → contain all its dependencies in its package
- Security → keep data safe from any external threat.
- Safety → shouldn't be hazardous or harmful to env.

→ Scalability :-

1) Vertical Scaling



→ adding/removing resources on a machine

→ Simpler, cheaper, and no need to change the code.

Disadv:- i) Single Point of failure

ii) Limit to hardware

iii) Downtime is high / required

2) Horizontal Scaling

→ adding/removing multiple machines → cluster

→ No single point of failure, no downtime required, no limit on no. of machines

Disadv:- i) Very expensive

ii) Skilled People are required to make changes

iii) Very complex

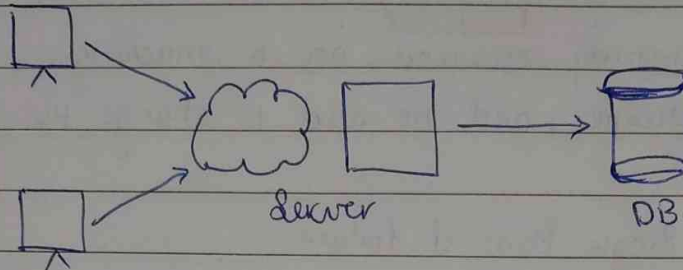
iv) Code changes are required.

→ VAPT (Vulnerability and Penetration Testing)

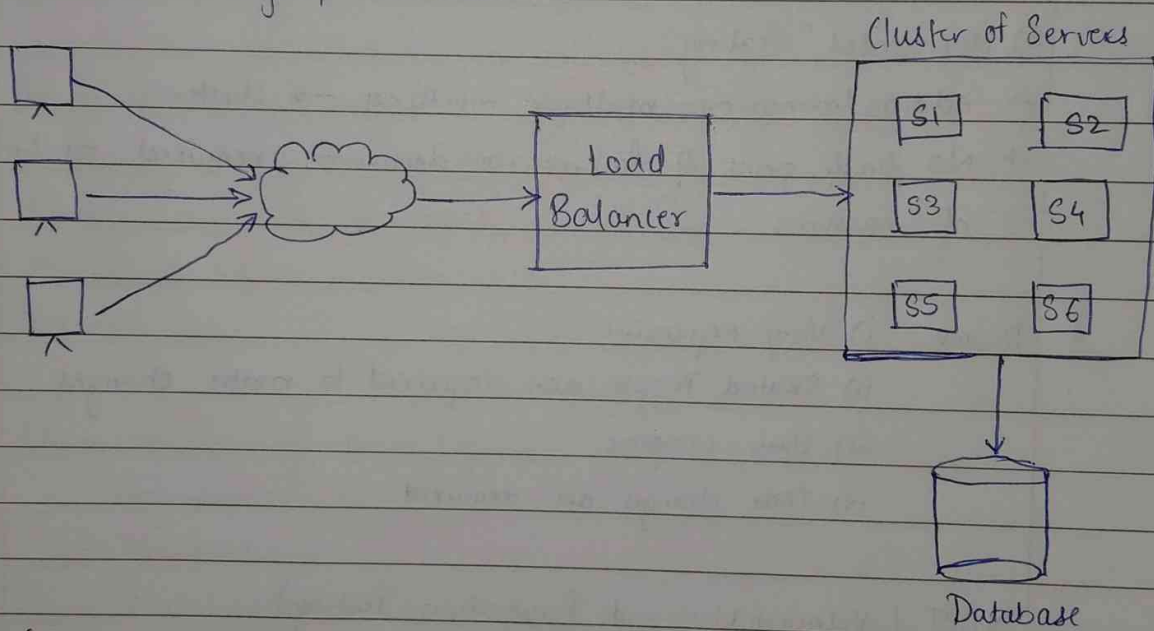
→ Transitional can be measured by :-

- Portability → software should show same functionality on diff. env & diff. platform
- Interoperability → ability of software to use information transparently
- Reusability → code
- Adaptability → ability to adapt the new changes in the environment.
→ Developer Environment :- Environment consisting of all the development tools required for development of software.

→ Staging Environment:- Environment used by testers to test application.

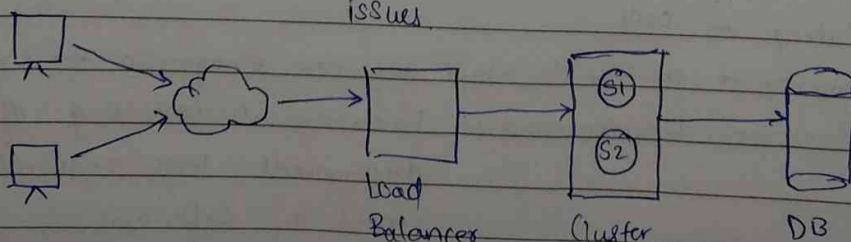


→ Production Environment:- accessible to end users
↳ Highly Available Env.



(Before, directly going to production environment from staging environment, the application goes into pre-production env.)

→ Pre-production Environment:- Used by tester to test the applications availability & server/cluster related issues.



* To go from one environment to another environment, we do Deployment.

→ Maintenance can be measured as on

- Maintainability :- easy to maintain by end user.
- Flexibility - flexible to any changes made to it
- Extensibility - create no problems when ^{test} adding no. of functions
- Testability - easy to ^{test} software
- Modularity - Should be highly modular, divided
- Scalability - Should be easy to upgrade.

→ SDLC (Software Development lifecycle)

Planning → Defining → Designing → Building → Testing → Deployment

→ a well-defined, structured sequence of stages in software engineering to develop the intended software product.

→ Aims to produce a high-quality software that

- meets or exceeds customer expectations
- reaches completion within times and cost estimates

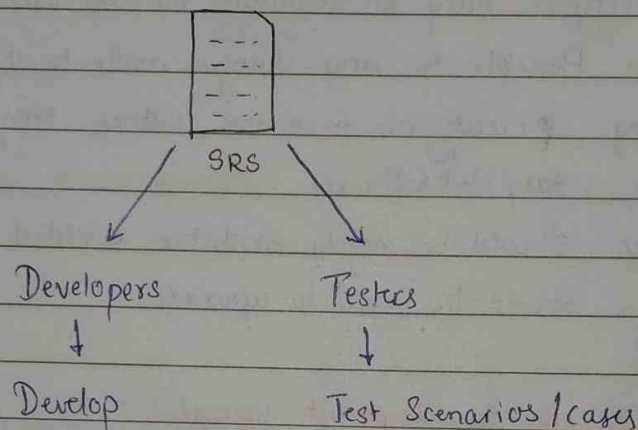
1) Planning (most fundamental stage in SDLC) → Requirement Gathering & Analysis

→ This info is used to plan basic project approach and conduct product feasibility, study in economical, operational & technical areas.

→ Planning for quality assurance requirements and identification of the risks associated with the project.

2) Defining

- done through an SRS (Software Requirement Specification) document which consist of the product requirements, to be designed & developed during the project cycle.



3) Designing

- Designing the product workflow / product screens, etc.
- Based on the requirements specified in SRS, more than one designing approach is ~~used~~ for the product architecture is proposed and documented in DDS - Design Document Specification.
- The internal design of all the modules of the proposed architecture should be clearly defined with the minutest of details in DDS.

4) Building or Developing the product

- actual development starts and the product is built.
- Code is generated as per DDS during this stage.

- _/_/_
- If design is performed in a detailed manner, code generation can be ~~managed~~ accomplished without much hassle.
 - Developers must follow developer guidelines defined by their ~~guidelines~~ organization and programming tools like compiler, interpreters, debuggers, etc. are used to generate the code.
 - Programming language is chosen with respect to the type of software developed.

5) Testing

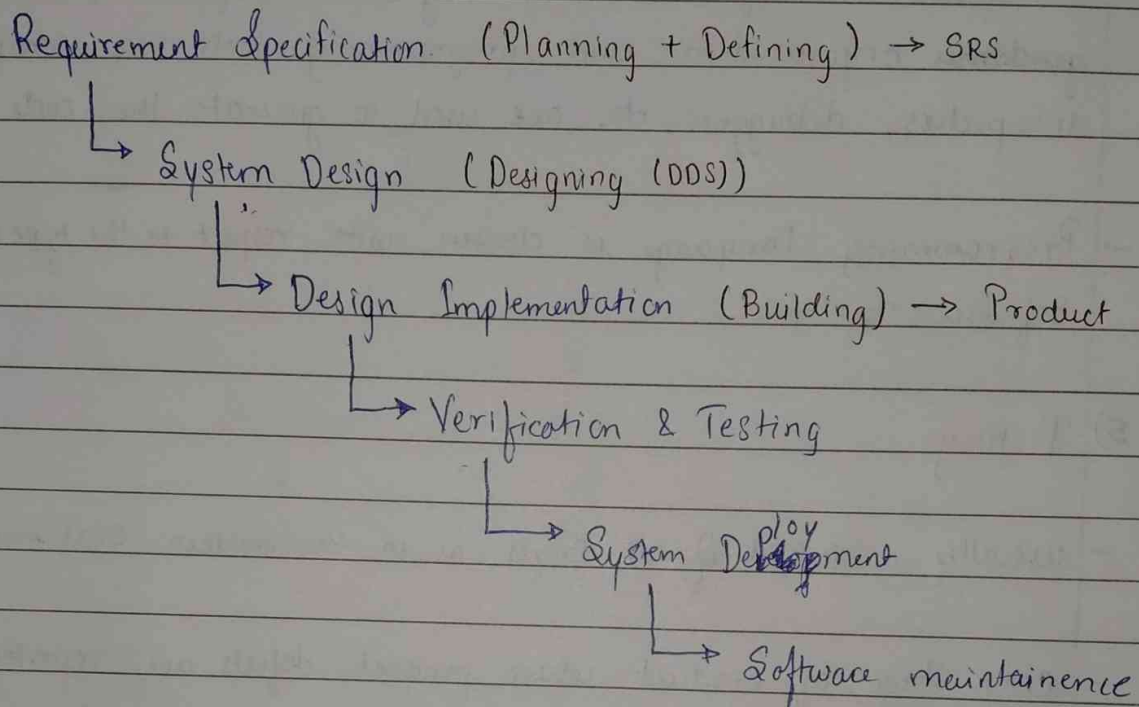
- usually a subset of all stages as in the modern SDLC models,
- Only stage of product where product defects are reported, tracked, fixed and retested, until the product reaches quality standards defined in the SRS.

6) Deployment

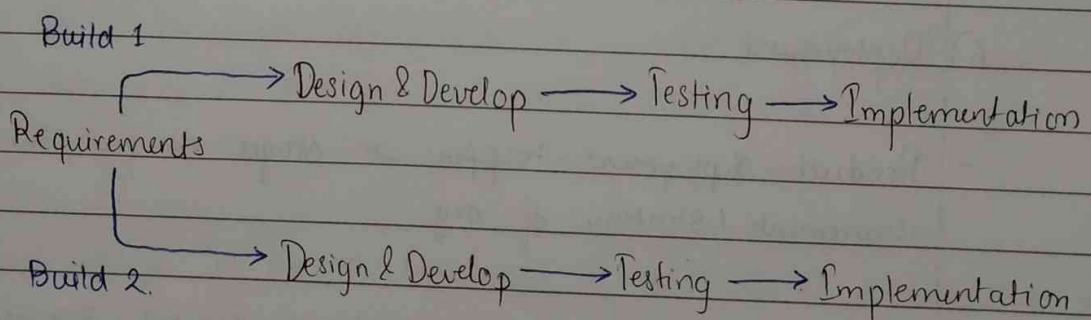
- Product deployment happens in stages as per the business requirements / strategy of org.
- Product may first be released in limited segment and tested in the real-business environment (UAT - user acceptance testing).
- Based on feedback, the product may be released or suggested enhancements in targeting market segments.

→ SDLC Models

① Waterfall Model



② Iterative Model



→ Product is developed module by module, and at the start of development entire requirements are not required.

→ This process is repeated until the whole product is developed.

* Agile Methodologies

Agile manifesto:-

- Individual and interactions over Processes and tools.
- Working Software over Comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Most popular Agile methods include

- Rational Unified Process.
- Scrum
- Crystal Clear.
- Extreme Programming
- Adaptive Software Development
- Feature Driven Development
- Dynamic Systems Development Method

* Scrum

- it's an agile way to manage project.
- it's a framework that facilitates processes amongst other things.
- Management framework with far reaching abilities to control and manage the iterations and increments in all project types.
- Incremental builds are delivered to customer in every two or three weeks time.
- Ideally used in projects where the requirement is rapidly changing.
- This is made up of a Scrum Team, individual roles, events, artefacts & rules.

→ Scrum Pillars

- 1) Transparency :- All parts of the process should be transparent open & honest for everyone to see.
- 2) Inspection :- Everything that is worked on during a sprint can be inspected by the team to make sure that it is achieving what it needs to.
- 3) Adaption :- If a member of the scrum team or a stakeholder notices that things are not going to plan, the team will need to change up what they're doing to ~~do~~ fix this ASAP.

→ Scrum Values

- 1) Courage :- Courage to do the right thing & work on tough problems
- 2) Focus :- Focus on the sprint & its goal.
- 3) Commitment :- Commitment to the team & Sprint goal.
- 4) Respect :- Respect for each other by helping people to learn things that you're good at and not judging things that others aren't good at.
- 5) Openness :- Be open and honest

* Sprint → Time Boxed Event (1-4 weeks)

__/__/__

→ How Scrum Pillars helps us ?

- 1) Self-Organization
- 2) Collaboration
- 3) Time-Boxing
- 4) Iterative Development

(Epic → Collection of Stories)

(Story → task / action to perform)

* Signin Epic

- Create a SignIn API
- Create a SignIn Page
- Connect frontend to backend

→ Scrum Roles

→ Scrum Artifacts

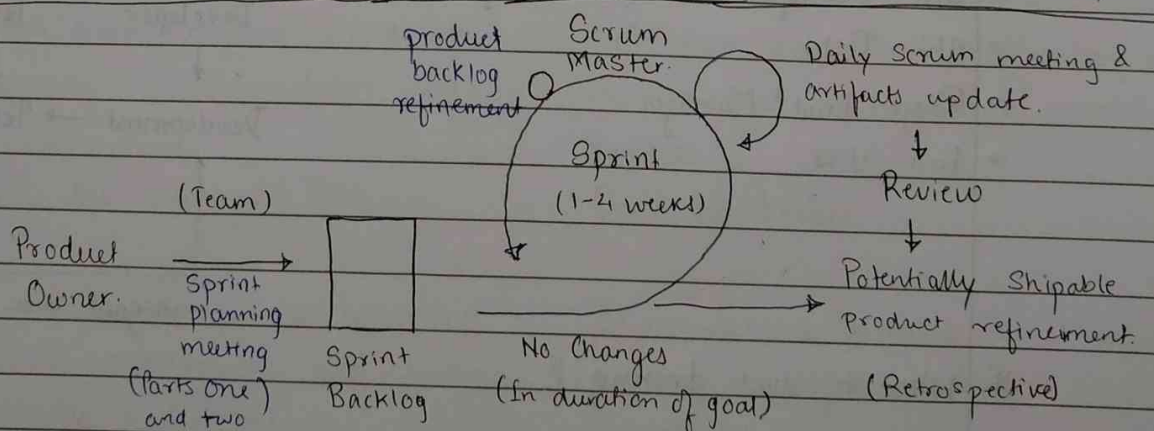
- ① Product Owner
- ② Scrum Master
- ③ Dev Team

① Product Backlog

→ Collectⁿ of all pending stories

② Sprint Backlog → Collectⁿ of stories to be completed within a sprint

③ Increment



Increment → version of a app with completed sprint backlog

→ Scrum Events

- ① Sprint Planning
- ② Daily Scrum
- ③ Sprint Review
- ④ Sprint Retrospective
- ⑤ Sprint.

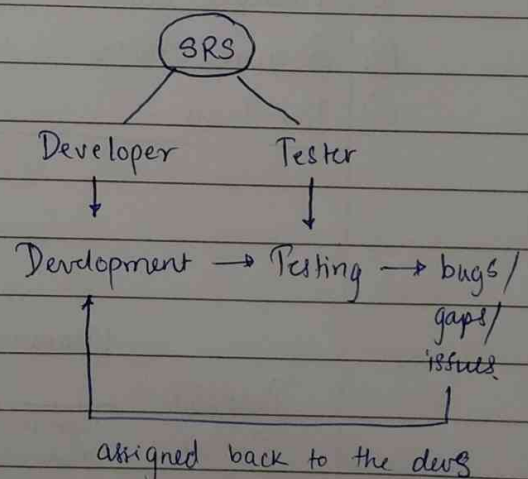
* Application Testing

Testing → Process of evaluating a system or its components with intent to find whether it satisfies the specified requirements in SRS or not.

A process of analyzing a software item to detect any differences between existing & required conditions and to evaluate the features of Software item.

Testing is done by

- S/w Developer → Unit Testing
- S/w Tester
- Project Lead / Manager
- End User



* When to start testing ?

Testing should be done at each and every phase of SDLC, i.e. during requirement gathering phase, the analysis, and verification of requirements (also considered as testing).

* unit → Function

Testing Performed by developer on completion of code is also considered as testing (Unit Testing)

* When to stop testing?

- Testing Deadlines
 - Completion of Test case execution
 - Completion of functional and code coverage to a point
 - Bug rate falls below a certain level and high-level bugs identified
 - Management Decision
- how much code is covered by unit testing
certain
↑
no

Verification vs Validation

- | | |
|---|---|
| - Addresses the concern:-
Are you building it right? | - Addresses the concern:-
Are you building the right thing? |
| - Ensures that the software system meets all the functionality | - Ensures the functionalities meet the intended behaviour |
| - Takes place first and includes the checking for documentation, code | - Validation occurs after verification and mainly involves the checking of overall Product |
| - Done by Developers | - Done by testers |
| - It has static activities, collecting reviews, walkthroughs and inspection to verify a software. | - It has dynamic activities, as it includes executing the software against the requirements |

PTO

* Quality Assurance vs Quality Control

- | | |
|---|---|
| - QA includes the activities that <u>ensure the implementation of processes</u> procedures and standards in context of verificat ⁿ of developed s/w & inter-
-ded reqs. | - QC includes activities that ensu
the verification of a developed s/w
with respect to documented
requirements |
| - Focuses on processes and rather than
conducting actual testing on system | - Focuses on actual testing by executing
the s/w with an aim to identify bug
through implementation of process |
| - Process-oriented activities | - Product-oriented activities |
| - Preventive activities | - Corrective-Process |
| - Subset of STLC | - Subset of QA |

→ Audit (Done internally)

- Systematic process to determine how the actual testing process is conducted within an organizatⁿ.
- it's an independent examinⁿ of processes involved during the testing of a s/w.

→ Inspection (Performed by external entity)

- A formal evaluation technique in which s/w requirements, designs or codes are examined in detail.
- Conducted by a group or a person other than the author to detect faults, violations of development standards, and other

* Types of Testing

1) Manual Testing

- includes testing software manually, i.e. without using any automated tools or scripts.
- Tester takes over the role of end-user and tests the software to identify any unexpected behaviour or bug.

diff stages of testing :-

- i) unit testing
- ii) integration testing
- iii) System testing
- iv) User acceptance testing

- Cheaper than automated testing and doesn't require any programming.
- Takes / Requires a lot of time.

2) Automated Testing

- Also known as test automation
- Tester writes scripts and uses another slw to test the product.
- Involves ~~no~~ automation of manual process

- Automation testing is used to re-run the test scenarios that were performed manually, quickly, repeatedly.

- Used to test

i) Regression

ii) Performance

iii) Stress Point

- It increases test coverage, improves accuracy, and saves time and money in comparison to manual testing.

→ When to automate

- Large and critical projects

- Projects that require testing same areas functionality

- Requirements not changing frequently

- Accessing the applicatⁿ for load and performance testing with many virtual ~~area~~ users.

- Stable slw w.r.t. manual testing

- Availability time.

→ How to automate?

- Identifying areas within a slw for automation

- Selection of appropriate tool for test

- Writing test scripts

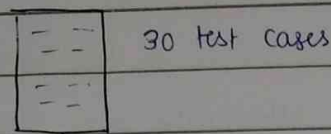
- Development of test suits

- Execution of scripts

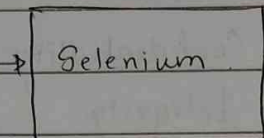
- Create result of reports

- Identify any potential bug / performance issues

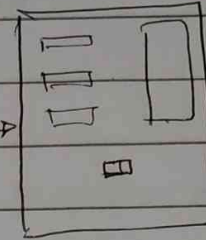
Collectⁿ of test cases
test Suits



Script



Automation Tool



Login

- Types
- i) Black box
 - ii) White box
 - iii) Gray box

→ Levels of Testing

① Functional (checks if applicatⁿ is functionally correct or not)

- type of black-box testing, i.e. based on specification of S/W that is to be tested
- it's conducted on a complete, integrated system to evaluate the system's compliance with its SRS.

Includes - Unit Testing

Integration Testing

System Testing

Regression Testing → (check if one bug leads to another bug)

Acceptance Testing → (testing done for acceptance of user/customer)

Alpha Testing (combiniⁿ of all 5)

Beta Testing (sample of small user test app)
→ (Pre-release testing)

② Non-Functional

- involves testing a software ~~from the~~ non-functional requirement perspective

- Includes:-
- Performance Testing
 - Usability Testing
 - Security Testing

↳ CIA Triad

- Confidentiality
- Integrity
- Availability
- Authentication
- Authorization
- Portability Testing

* Selenium

- UI Testing
- Web Scraping
- Open source and portable automated testing tool for testing web application.
- Capable of operating across diff. browsers & OS's.

Tools → Selenium IDE

Selenium RC

Selenium WebDriver

Selenium Grid.

Advantages :-

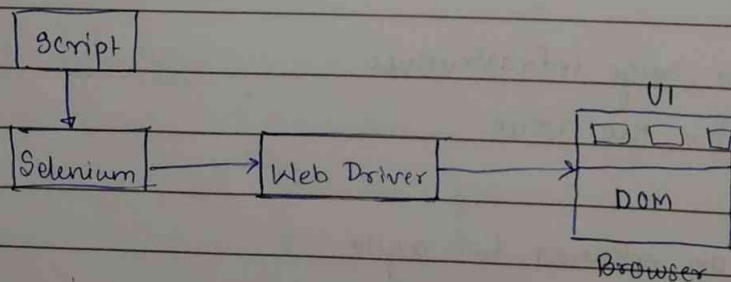
- an open source tool
- can be extended for various technologies that expose DOM
- Has capabilities to execute scripts across diff browsers
- Can execute scripts on various operating systems
- Support mobile devices
- Can execute tests in 11d with the use of Selenium Grid.

Disadvantages

- Support only web-based applications
- No feature such as Object repository / Recovery Scenario
- Cannot access controls within the browser
- No Default test report generation
- For parameterization, user has to rely on the programming language

→ Web Driver (Imp)

- it's a tool for automating ~~testing~~ web applications
- popularly known as Selenium 2.0
- Interacts directly with the browser, and uses the browser's engine to control it
- It is faster, as it interacts directly with the browser
- It can support headless execution
 - ↳ means without opening the browser GUI or Tab.



→ Locator

Element locators help Selenium to identify the HTML element the command refers to.

(Read remaining from PDF)

DevOps

Ops Team Responsibilities :-

- Infra. Mgmt.
- Security & Compliance
- Deployment & Release Mgmt.
- Monitor, logging & Alerting
- Incident Mgmt. & Troubleshooting
- Cloud & Cost Optimizatⁿ
- Backup & Disaster Recovery
- Collaboration & Support
- Performance & Capacity planning

Challenges Faced:-

- Uptime
- Configure the huge infrastructure
- Diagnose & fix the issue.

→ Devops is an extension to agile.

- Problems

- Managing & tracking changes in code is difficult. (SCM Tools)
- Incremental builds are difficult to manage, test and deploy. (CI/CD)
- (Selenium) - Manual testing & Deployment of various components/modules takes time.
- (Ansible) - Ensuring consistency, adaptability and scalability across environments is very difficult task.
- Main goal is to deliver software faster, more reliably, and with higher quality - by automating & integrating the processes of software development and ~~IT~~ IT Operations.

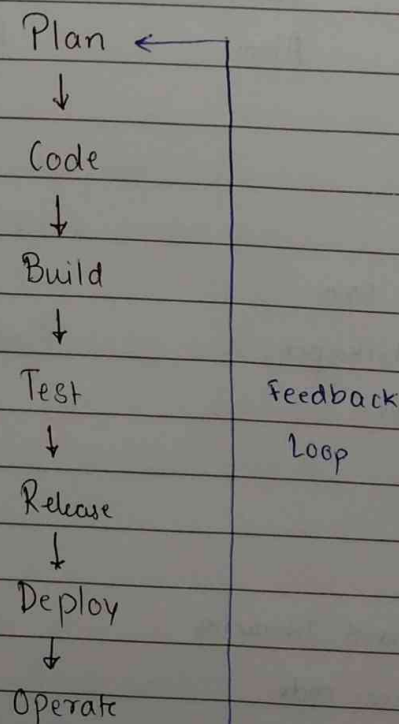
→ Goals of DevOps

- Faster Delivery → Shorten time from idea - deployment
- Better Collaboration → Break Silos b/w Dev & Ops
- Automation → Reduce manual errors and repetitive work
- Reliability → Ensure systems are stable & Scalable.
- Continuous Improvement → Use feedback to improve process

→ Reasons to use DevOps

- Predictability
- Reproducibility (rollback)
- Greater Quality
- Reduced Risk. (more secure, less defects)
- Resiliency (more stable, secure)

Devops Lifecycle



1) Plan

- here we plan, track, visualize and summarize your project before starting to working on it.

Simple Tools :-

text files

Excel Sheets

Cloud Tools :-

Google Drive

MS One Drive

Special Tools :-

Jira

BitScrum

Trello

2) Code

- here developer writes the code using programming lang.

Languages :-

C/C++ / Java

Rust

JavaScript

TypeScript

Editors :-

Vim

VS Code

Sublime Text

Atom

IDEs :-

MS Visual Studio

WebStorm

IntelliJ Idea

PyCharm

Package Manager

npm

yarn

pnpm

bun

SCM tools :-

Local VCS → RCS

Centralized VCS → CVS, SVN

Distributed VCS → git, bitkeeper
bazaar

3) Build

- Integrating required libraries
- Compiling the source code
- Creating the build artifacts

→ Source code → Building Tool → Package

Build Tools:-

Ant Xcodebuild

Maven

Gradle

4) Test

- Process of executing automated tests
- The goal here is to get the feedback about the changes as quick as possible

Unit Testing Tools

java → JUnit

.NET → NUnit

python → PyUnit

JavaScript → Jasmine/Jest

Web UI Testing

Selenium

Cypress

Performance Testing

Jmeter

WinRunner

LoadRunner

5) Release

- This phase helps to integrate code into a shared repository using which you can detect and locate errors quickly & easily

CI Tools

CD Tools

CI/CD Tools

Travis CI

ArgoCD

Jenkins

GitLab CI

GitHub Actions

Circle CI

Bamboo

6) Deploy

- Manage and maintain development & deployment of software systems and server

Deployment Strategies

① Traditional Deployment

↳ using physical machines

+

deprecated

② Virtualized Deployment

↳ using virtual machines

→ On Premise (Private)

- Virtual Box
- VMware
- Qemu
- BlueStack

→ On Cloud

- AWS (EC2)
- GCP (Virtual machine)
- Azure (Virtual machine)

③ Containerized Deployment

↳ using containers

→ Container Runtimes

- docker, containerd.
- Podman, LXC, LXD

→ Container Orchestration

- K8S, Apache Marathon,
- Keda, docker swarm

7) Operate

- Stage where the updated systems get operated

infra creation tools

Terraform (Hashicorp)

AWS Cloud formation

Vagrant - local vm mgmt.

infra. config tools

Puppet

chef

Ansible

8) Monitor

- ensures application is performing as expected and the environment is stable.
- Quickly determines when a service is unavailable & understand the underlying cause.

Monitoring Tools

ELK (elasticsearch, logstash, kibana)

Prometheus

Grafana

NewRelic

Nagios

Datadog

Splunk

(Terminologies)

(CI/CD pipeline)

Continuous
Integration

Integration

(Release)

Continuous
Delivery

(Automatic)

Deploy → Continuous Deployment

Operate → Continuous Infra. Creation &

Monitor Continuous Configuration

↓

Continuous Monitoring

continuous } Plan
Coding } Code

Continuous Building → Build

Continuous Testing → Test

Be an excellent
Sysadmin

Soft Skills

Deploy Virtualization

~~Automation~~

Automation Tools

DevOps

Hands-on experience
in network & storage

Software Testing
knowledge

Introduction to
Coding

IT Security

* Virtualization

- Virtualization uses software that simulates hardware functionality in order to create a virtual system.
- allows IT organizations to operate multiple OS, more than one virtual system applications on a single server

Types :-

- Network
- Desktop
- OS
- Storage
- Application
- Data
- Hardware

→ Hardware Virtualization

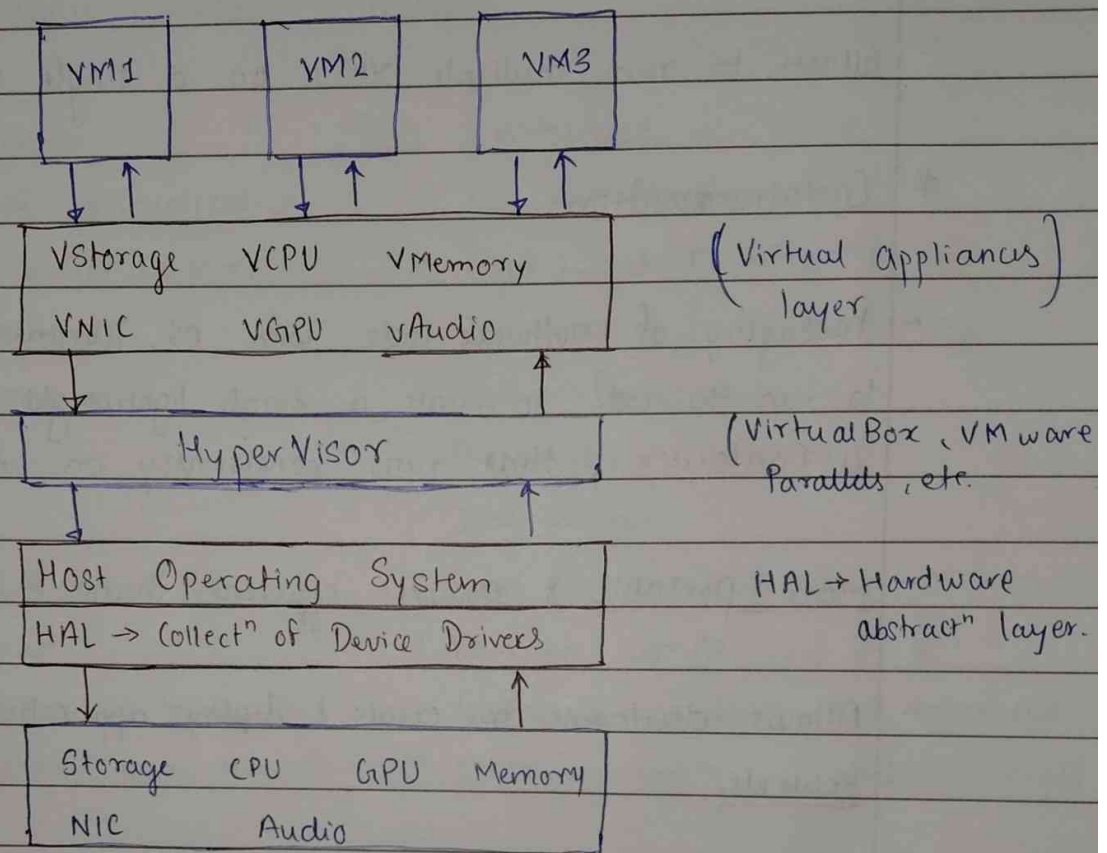
- Process of masking the hardware resources like
 - CPU
 - Storage
 - Memory
- Process of creating machines

Types of Hardware virtualization.

i) Type I

ii) Type II

Type II :-



Virtual Machine :-

application.

Runtime, libs,
binaries, etc.

Guest OS

Virtual H/W

Type II also known as Hosted Virtualization.

→ In type-I, Hypervisor and HOST OS layer is combined into one layer, to use the machine to its fullest, by having full access to the underlying Hardware

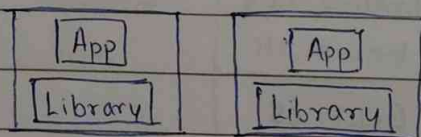
* Virtualized Deployment

- allows to run multiple VMs on a single server.

* Containerization

- Packaging of software code with OS, libraries, dependencies required to run the code to create a single lightweight executable is called a Container, that runs consistently on any infrastructure.
- More portable & resource efficient than VMs.
- Allows developers to create & deploy applications Faster & more securely.
- it eliminates the problem by bundling together the applicatⁿ code, related config files, libraries and dependencies required for it to run.

* Container Deployment



Container Runtime

Operating System

Hardware

→ Containers are considered lightweight

→ Container has its own file system, CPU, memory, process space & more

→ As they are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions

//_

→ Containers have process level isolation which makes them less secure than virtual machines.

<u>Imp</u> *	Virtual Machine	Containers
	Hardware level Virtualization	- OS virtualization.
	Heavyweight (bigger in size)	- Lightweight (smaller in size)
	Slow provisioning	- Real-time & fast Provisioning
	Limited Performance	- Native Performance.
	Fully isolated	- Process-level isolation
	More Secure	- Less Secure
	Each VM has separate OS	- Each container can share OS resources
	Boots in minutes	- Boots in seconds
	Can be easily moved to new OS	- Containers are destroyed & recreated
	Creating VM takes longer time	- Containers can be created in seconds.

Docker

- Open Source platform that enables developers to build, deploy, run and manage

* Why docker ?

- Provides consistent computing environment
- lightweight compared to VM.

* Docker Architecture

Docker Daemon - Continuous running process (dockerd)
- Manages the container

REST APIs - used to communicate with
+ docker daemon

Client (docker) - Provides command line interface
- used to perform all the tasks.

Images Volumes Network Container.

Client

Rest APIs

Docker Daemon.

* Docker Image:-

- it's a read-only template that contains everything related to run an application - include the code, runtime, libraries, environment variables and configuration files.
- Images are blueprint for creating containers
- Acts like a snapshot of a file system and its configuration.
- Docker images are build using layered file system (Union FS)
 - Base Layer
 - Intermediate Layer
 -

- * Docker Container

- running instance of an image
- Contains one or more running processes
- It's a self-contained environment
- Container can't modify the image from which it is created
- it wraps up an application into its own isolated box ("application" running inside a container has no knowledge of any other applications running / exist outside the container)
- Consists of → Application Code Networking
 Dependencies Volume.

→ Containers are stored under `/var/lib/docker`

* On ubuntu, apache uses `/var/www/html` as `htdocs` directory
(hyper text docs)

→ To get ip address of our machine

> `ifconfig`

> `ip address show`

> `ip a` → short form of above

→ To add a mapping of ip → name/domain name.

i) go to `/etc/hosts` (use `Sudo`)
↑ file

ii) add a mapping inside that file

▶ `172.18.11.10 mywebsite.com`

* How the container works?

(inspect container?)

`hub.docker.com`

`httpd nginx mongo`

`alpine node postgres`

↓
`docker container inspect cont-name`

`docker image pull httpd`

`httpd`

`docker run -d --name myhttpd httpd`

(80)
`httpd`

`index.html`

`/usr/local/apache/htdocs`

`/var/www/html`

Container Runtime - Docker

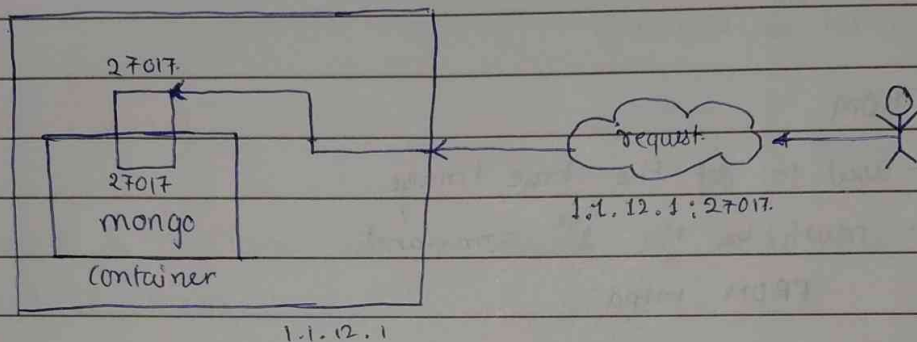
Linux

→ Container can run only one application at a time

___/___/___

Port forwarding

- Forward a container's port to a host's port



* request hits host's port 27017 & then it gets redirected to container's port 27017.

`docker run -d -p host:container <image>`

→ Executing commands inside a terminal of a container

`> docker container exec -it <container_name> <command>`

-i → interactive

-t → terminal

→ run a container in detached with a volume attached

`docker run -d --name mysql -p 3306:3306 -v <volume_name>:<mount_point> <image>`

* Custom Images

- image created to host a custom application

Instructions:-

- FROM

- used to set the base image
- must be the 1st command

FROM httpd

- COPY

- used to copy file(s) from local machine to image.

COPY <local-file> <image-dest>

- EXPOSE

- used to export a port which is used by application

EXPOSE 3000

- WORKDIR

- used to set working directory

WORKDIR /src

- CMD

- used to execute command at start of a container
- only one CMD is allowed per image
- must be the last command of dockerfile

CMD ["node", "index.js"]

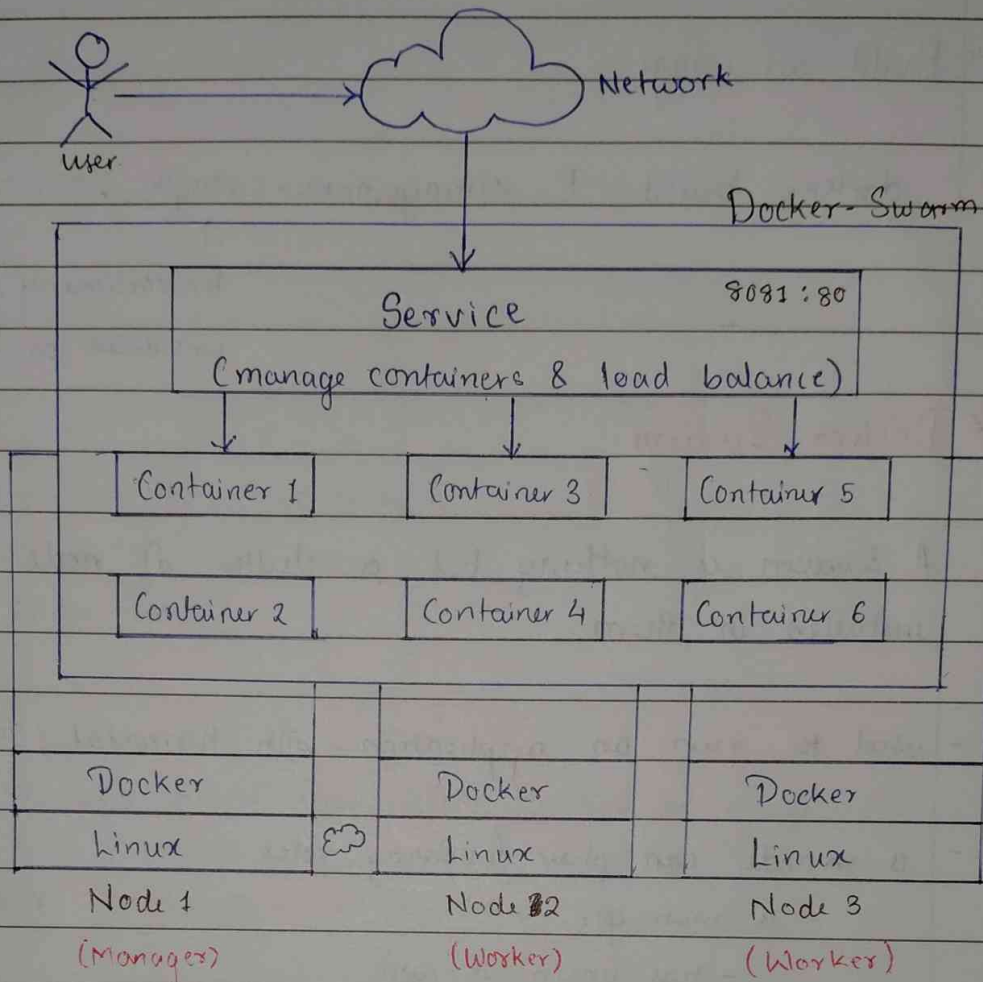
→ Build an image:-

```
docker build -t <image-name>:<tag> .
```

↑
for versioning, if not specified then
considered as latest

* Docker Swarm

- A Swarm is nothing but a cluster of node having docker installed in them.
- used to run an application with horizontal scaling.
- a node can play following roles
 - a manager
 - has brain or apis
 - controls entire cluster.
 - a worker
 - has more power
 - runs the container
- Types of cluster
 - ① Single node cluster
 - one machine playing the role of manager
 - the manager will control the cluster as well as run the containers
 - generally used while developing / testing an application.
 - ② Multi node cluster
 - multiple nodes playing manager and worker roles
 - preferred in production



- Single-manager cluster
- used in pre-prod env

- multi-manager cluster
- used in prod env.

- * If manager node goes down, then there are always 2 sub-cluster manager (above two)
- * Only one manager is considered to be a leader. Leader's decision is final.
- * Port-forwarding is done at service level instead of doing at container level.

→ Check if machine is part of swarm or not.

> docker system info | grep Swarm or
> docker system info

→ initialize a swarm

> docker swarm init

→ get list of nodes

> docker node ls

→ get list of services

> docker service ls

→ create a service.

> docker service create --name ^{Service-name} httpd ^{image} httpd

→ get list of containers managed by service

> docker service ps httpd

→ Start multiple containers / scale a service

> docker service scale httpd=10

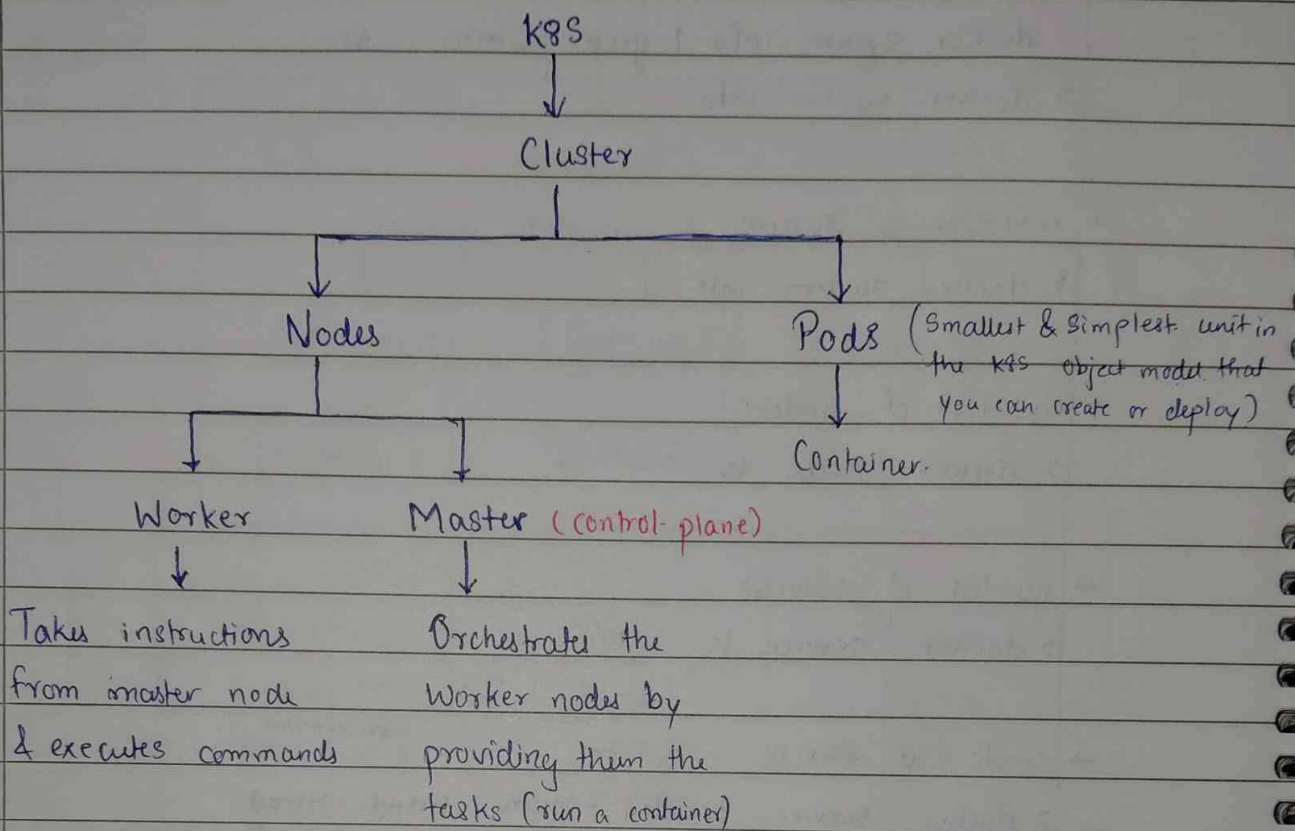
→ remove a service

> docker service rm <service-name>

→ Service with port forwarding & 10 replicas.

> docker service create --name ^{Service-name} httpd -p 8081:80 --replicas 10 httpd

Kubernetes



Architecture of Master Node

- API Server → ~~acts as~~ api for user to interact with cluster
- etcd
- KubeScheduler
- KubeControllerManager → infinite loop, which checks ~~everytime~~ ^{be} if any tasks are pending to, forwarded to worker Node.

Architecture of Worker Node

- kubeProxy (~~network proxy, running on each node~~) in k8 cluster, to talk to pods)
- Kubelet → responsible for checking if there is a process for that worker node
- Container Runtime (~~Podman~~, Podman) present in master node.
Containerd,

Pods :- A pod can run ~~multiple~~ multiple containers to simplify the sharing of storage/network, but most of the time a pod mostly runs a single container at a time.

* Container Orchestration

- it's all about managing the lifecycle of containers, especially
- used by software teams to control & automate many tasks
 - provisioning and deployment of container.
 - redundancy and availability of containers
 - Scaling up or removing containers to spread the load across host infra.
 - Allocation of resources b/w containers
 - Load balancing of service discovery b/w containers
 - Health monitoring of containers & hosts.

→ Tools

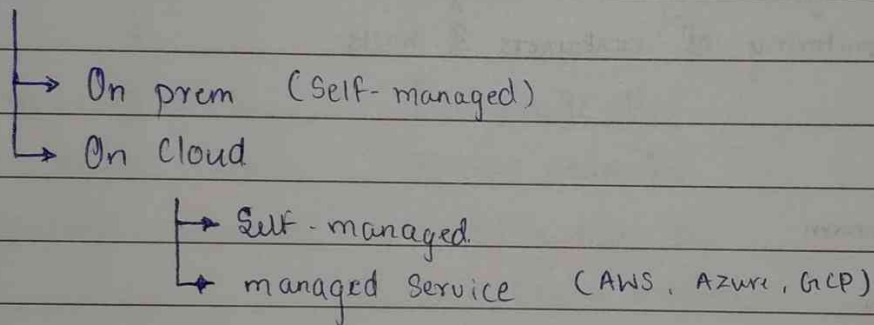
- Docker Swarm
 - Kubernetes
 - Mesos
 - Marathon
-
- Each container sends a packet to manager/master node, ~~to~~ after a fixed interval, to ensure the container is monitored continuously. if a container stops sending signal (heartbeat signal), the master / manager start a new container to maintain availability.
-
- Service is an object that manages the load and containers.
 - Manager (Master node) creates the service and it's a role.

Kubernetes (K8S)

- Portable, extensible, open-source platform for managing containerized workloads & services.
- Facilitates both declarative configuration & automation.
- It has a large, rapidly growing ecosystem

- ① declarative config → Yml or Yaml files, i.e. manifest files
- ② automation → CLI

installation



* Kubernetes Cluster

- When you deploy Kubernetes, you get a cluster.
- A cluster is a set of machines called as Node, that run containerized application managed by Kubernetes.
- A cluster has at least one Master and one Worker node.
- A worker node hosts the pods that are the components of the application (frontend, backend, db, websocket)
- The master node manages the worker nodes & pods in the cluster
- Multiple master nodes are used to provide a cluster with failover and high availability.

* K8s Cluster

* Single Node Cluster

- Simulated / Virtual Cluster
- configured by tool → minikube
- used only for learning

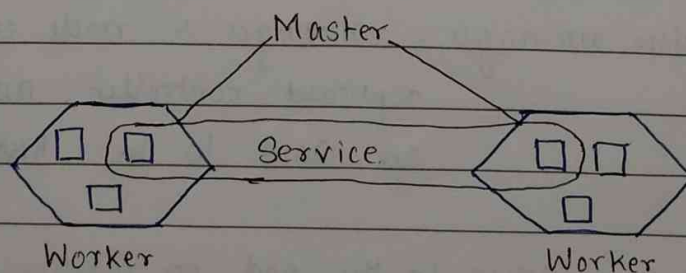
* Multi Node Cluster

i) Single master cluster

- One master & one or more worker nodes
- used in dev or testing environment.

ii) Multi master cluster

- multiple masters with multiple worker nodes
- Highly Available Cluster
- preferred in prod. environment
- From multiple masters, only one is elected as Leader (using raft-consensus algo.)



* Kube-Proxy :- a proxy server which provides the internal & external communication of worker node with master node as well as the external env., by providing the ip-addresses, network rules, etc.

* Kubelet :- an agent which facilitates the master node by providing the info. about worker node, ~~about~~ about its status (up/down) and health checks are done by master node.

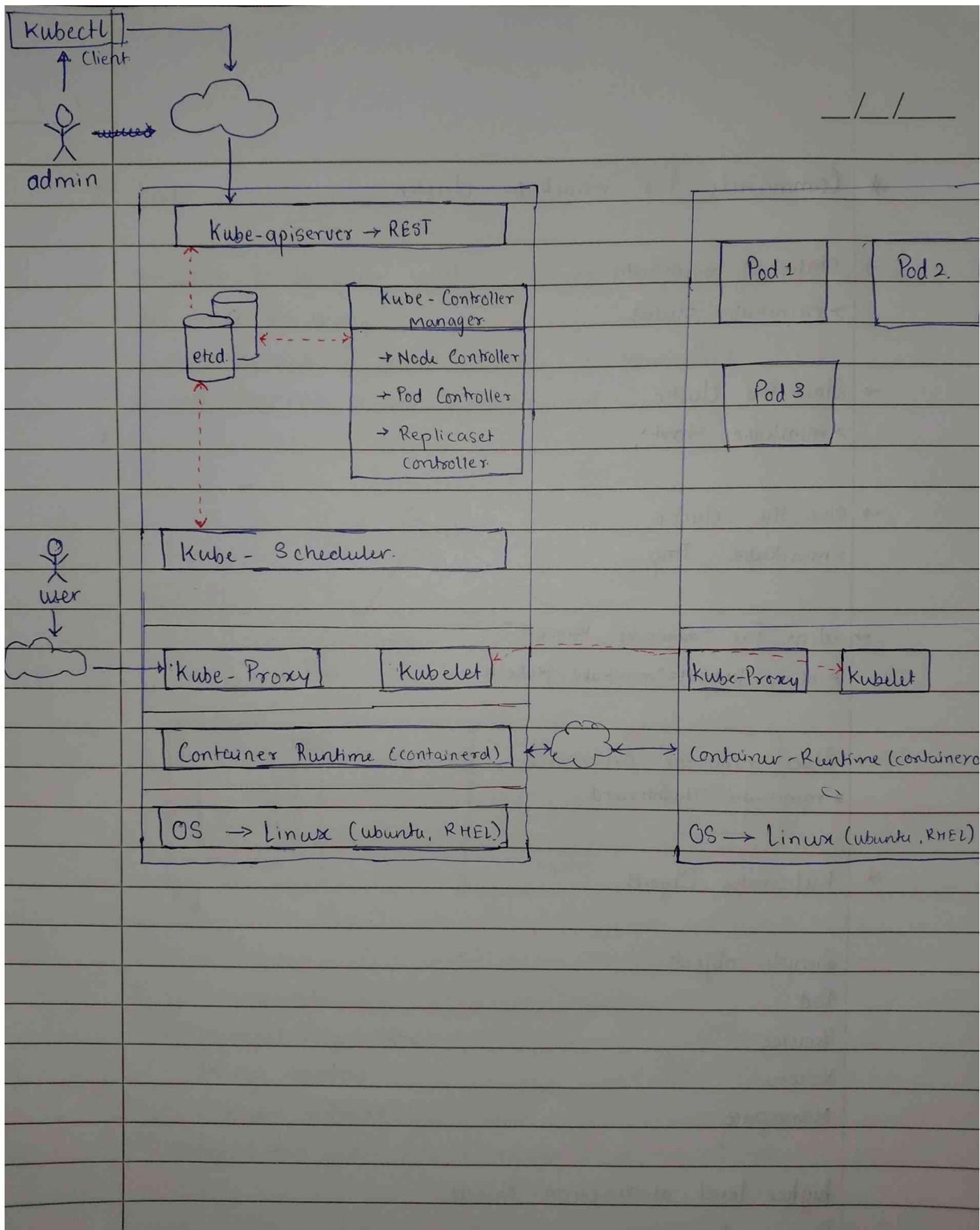
* Container- Runtime :- used to run containers, i.e. provides an environment.

Brain) * kube-apiserver :- ~~Provides~~ Provides api's exposed to the user to get instructions and execute them

* etcd :- Stores the information about worker nodes, i.e. what containers ^{Pods} are running, how many, how much resources available, etc.

* Kube-controller-manager :- manages a node controller, pod controller, replicaset controller, all of them together are said to be kube controller manager.

* Kube-Scheduler :- schedule the pod on worker node which has enough resources, by evaluating the info from etcd.



* Commands for minikube cluster

→ status of minikube.

> minikube status

→ Start the cluster

> minikube start

→ Stop the cluster

> minikube stop

→ alias for "minikube kubectl"

> alias kubectl="minikube kubectl --"

→ Start minikube dashboard

> minikube dashboard

* Kubernetes Objects

- Simple objects

Pod

Service

Volume

Namespace

- higher level abstraction objects

Deployment

Replicaset

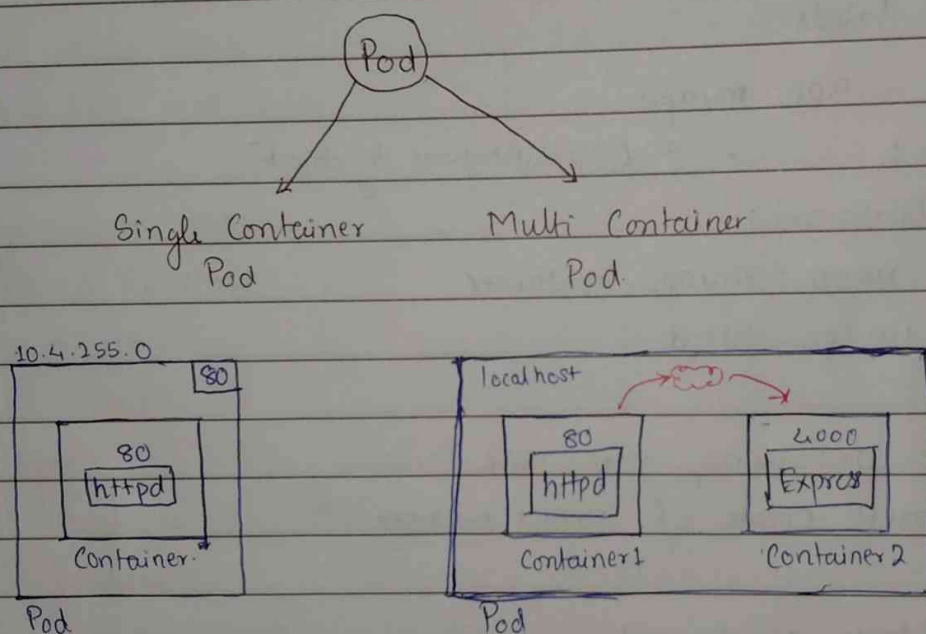
Statefulset

Daemonset

Job

* Pod

- smallest & simplest unit in a k8s application, that you can create or deploy.
- a pod represents a process running on your cluster.
- a pod represents a unit of deployment.



- a pod encapsulates
 - an application container
 - storage resources
 - a unique network IP
 - options that govern how a container should run. (config)

* YAML to create pod

apiVersion: v1 --> (version of k8s) (basic object: v1
high level obj.: apps/v1)

kind: Pod --> (type of object) (Pod, Deployment, Service, etc).

metadata:
~~image~~ --> (info about the obj. to be created)

name: myapp-pod (e.g. name, labels, namespace, etc)

labels:

app: myapp

Spec: --> (Specifications of object)

Containers:

- name: myapp-container

image: httpd.

→ Create pod using yaml file

> kubectl create -f <yaml-file-name>

→ get details of a pod

> kubectl describe pod <pod-name>

→ delete the pod

> kubectl delete pod <pod-name>

→ ssh into minikube (will give access to VM, i.e. running the cluster)

> minikube ssh

→ get the list of pods with more options

> kubectl get pods -o wide

* Replicaset

- ensures a specified number of pod replicas are running at a given time.
- it makes sure that a pod or a homogeneous set of pods is always up and available.
- if too many pods, replicaset terminates extra pods.
- if too few pods, replicaset starts more pods.
- pods maintained by replicaset are automatically replaced if they fail, go down.

apiVersion: apps/v1

kind: Replicaset

metadata:

name:

Spec:

desired count.

replicas: 3

Selector:

matchLabels:

app: nginx

template:

metadata:

name: nginx

labels:

app: nginx

Spec:

Containers:

- name: nginx

image: nginx

ports:

- containerPort: 80

must be
same.

→ get list of replicaset

> kubectl get replicaset

> kubectl get rs

→ Start the replicaset

> kubectl create -f rs1.yml

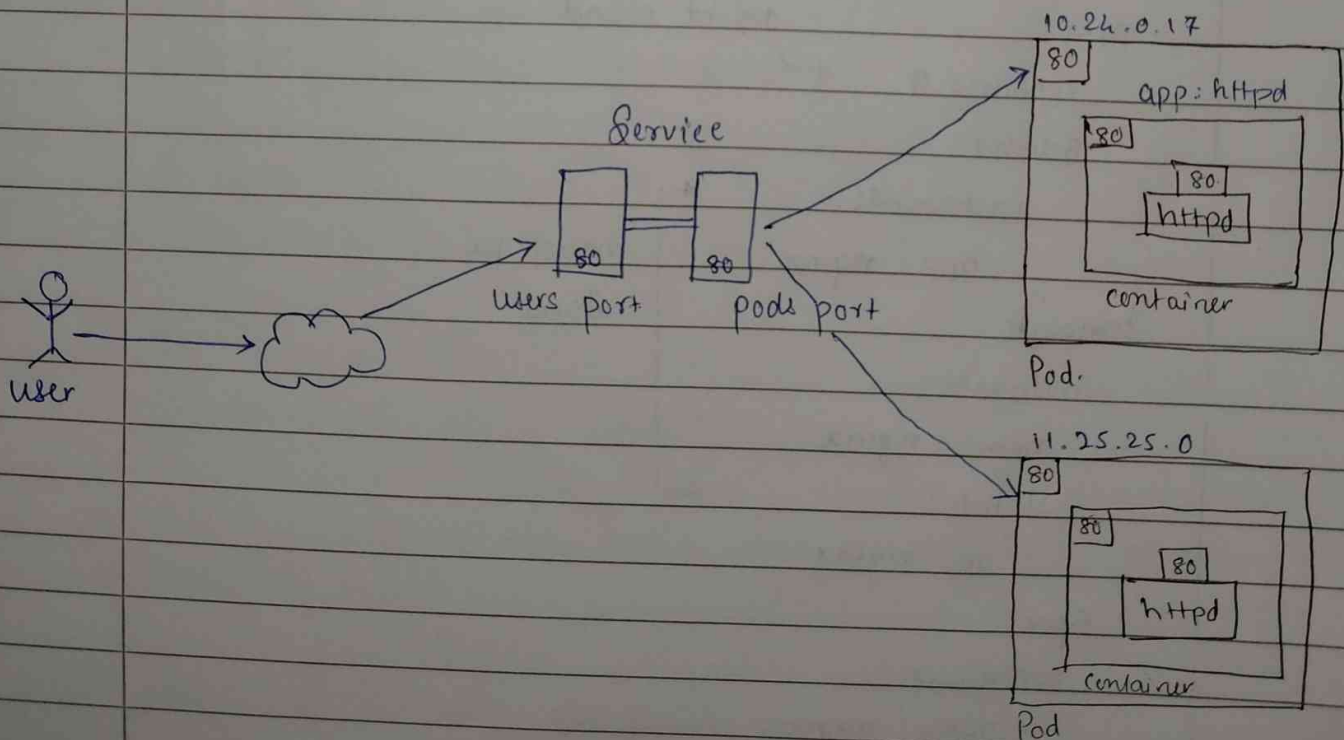
↑ yml file name.

→ delete a replicaset

> kubectl delete rs <replicaset-name>

* Service

- an abstract way to expose an application running on a set of pods as a network service.



yml Syntax

apiversion: V1

kind: Service

metadata:

name: nginx

spec:

selector:

app: nginx

ports:

- name: http

port: 80

targetport: 80

→ get the service

> kubectl get service

→ Create the service

> kubectl create -f <service-yml-file>

→ delete a service

kubectl delete service <service-name>

→ expose a service from minikube vm

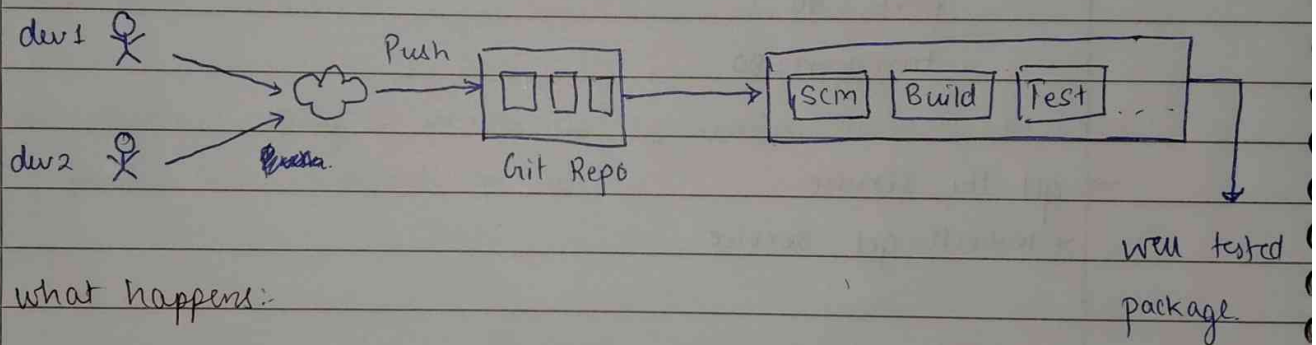
minikube service <service-name>

CI / CD Pipeline

- Pipeline → Sequence of stages
- CI → Continuous Integration
- CD → Continuous Deployment / Delivery

* CI (done in release stage)

- practice of automatically integrating code changes from multiple developers in a single shared repository frequently - often several times a day.



what happens:-

- Devs. push code to a VCS (Git)
- Automated tests run to check if new code breaks anything
- Code is automatically compiled/build.
- Teams get immediate feedback if something goes wrong.

git → main/master branch,

∴ it contains the

- working code
- well tested code
- non-breaking code
- latest code having latest features.

- _/_/_
- Core principles of CI
 - Single Source Repository (Shared)
 - Frequent Commits
 - Automated Build
 - Self-Testing Build
 - * - unit tests (Test individual functions/components)
 - integration tests (Test how components work together)
 - static code analysis (check code quality & standards)
 - Security scans (identify vulnerabilities)
 - Fast Builds
 - Test in Production-like Environment
 - Early access to latest build.
 - Fix broken build immediately.

* The CI Workflow

- ① Developers works locally
 - pulls latest code & from main branch
 - create feature branch & make changes & write tests
 - Run tests locally to ensure everything works.
- ② Commit and Push
 - Developers commit changes with a descriptive message
 - push code to a central repository
- ③ CI server detects changes
 - CI server monitors the repo
 - Detects new commit via webhooks / polling

④ Checkout code

- CI server pulls latest code from repo
- creates a clean build environment

⑤ Build Process

- compiles the source code
- resolve dependencies
- Creates Build artifacts (executables, containers, packages)

⑥ Run Tests

- Execute automated test suites
- Generates test reports & code coverage metrics
- may run multiple tests in parallel.

⑦ Code Quality checks

- Run linters to check coding standards
- Perform static code analysis
- Checks for security vulnerabilities
- Verifies code coverage meets thresholds

⑧ Generate Artifacts

- If all checks pass, create deployable artifact
- Tag the build with version no.
- Store artifacts in a repo.

⑨ Notification

- Send results to the team (email, slack, ...)
- updates build status badges
- Green = Success, Red = Failed.

⑩ Feedback Loop

- If success :- Developer continues with new task.
- If failed :- Developer investigates & fix bugs.

* Key CI components

① VCS → Git, Subversion, Mercurian

② CI Server / Platform → Jenkins, GitHub Actions, Circle CI

③ Build Tools → Maven, Gradle (Java)

npm, yarn (JavaScript)

MSBuild (.NET)

Make, CMake (C/C++)

pip (python)

④ Testing Framework → JUnit, TestNG (Java)

Jest, Cypress (JavaScript)

pytest (C)

⑤ Code Quality Tools → SonarQube, OWASP, etc.

* Benefits of CI

① Reduced Integration Risk

② Higher Code Quality

③ Faster feedback

④ Improved Collaboration

⑤ Reduced Manual Testing

* CD

Continuous Delivery :- Code changes are automatically built, tested & prepared for release to production, but deployment requires manual approval. → Configured for prod. env.

Continuous Deployment :- Every change that passes all automated tests is automatically released to production without human intervention.
↓
any env
↓
automatic deployment → all internal environments

* Continuous Delivery

- Manual gate before production deployment
- Human decides when to release
- Business decides the release schedule.
- Suitable when releases need coordination.

Flow:- Code → Build → Test → Stage → [Approval] → Production.

* Continuous Deployment

- Fully automated pipeline to production
- Every successful build goes live automatically
-

Core Principles in CD

- Automate Everything
- Everyone is Responsible
- Done means released
- Work in Small Batches
- Build Quality In

* CD pipeline Stages

① Source Control

code - committed to VCS, it triggers the pipeline

② Build Automation

- Compile source code
- (whole CI)

③ Automated Testing

④ Staging / Pre-Production Deployment

⑤ Production Deployment

* Key Tech and Tools in CD

① Infrastructure as code (IAC)

- Terraform, Ansible, CloudFormation

② Container Orchestration

- Kubernetes, Docker swarm, AWS EKS, Azure AKS

③ Deployment Tools

- Helm, ArgoCD, Flux, Spinnaker

④ Configuration Management

- Consul, etcd, Spring Cloud Config, AWS system manager

⑤ Artifact Repositories

- DockerHub, Amazon ECR, Artifactory, Nexus, Github Packages

* Diff b/w Monolithic and Micro-Service Architecture

⑥ Monitoring & Observability

- Prometheus + Grafana, Datadog, NewRelic, DynaTrace, ELK stack, Splunk

* Jenkins

- an open-source automation server that enables developers to build, test & deploy software reliably & efficiently.

- Strengths

- Free and Open Source
- Highly Extensible
- Platform Agnostic
- Technology Agnostic
- Large Community
- Self-Hosted

- Weakness

- Steep learning curve
- Maintenance Overhead
- UI/UX
- Plugin management
- Scaling challenges

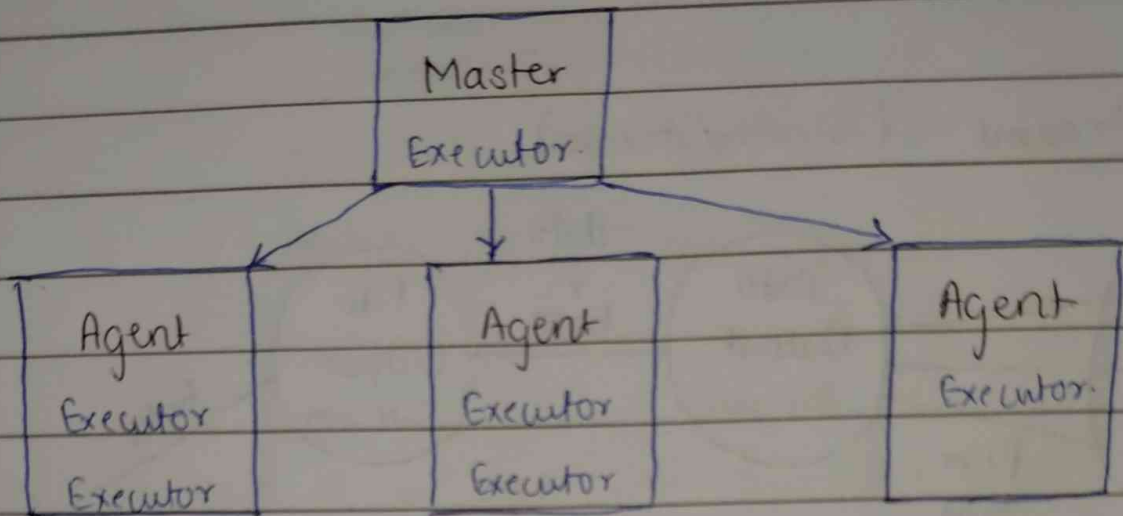
* Jenkins Architecture

1) Jenkins Master (Controller)

- main jenkins server
- manages the web interface
- Schedule build jobs
- monitors agents
- Distributes build to agents.
- Records and presents build history

2) Jenkins Agent

- Machines that execute build jobs
 - Receive instruction from master
 - Execute build steps
 - Report results back to master
 - can be static or dynamic.
- Executors
 - individual build slots on master or agent



* Cronjob

hour month
* * * * *
↑ ↑ ↑
minute day weekday